2025年度　修 士 論 文

複数ソフトウェアの修正履歴の混合による コーディング規約違反の修正予測手法

2026年2月

和歌山大学大学院システム工学研究科

学生番号: S2420033

亀岡 令

Prediction Approach of Coding Standard Violation Fixes by Combining Change Histories from Multiple Projects

by

Ryo Kameoka

Master's Thesis

Graduate School of Systems Engineering

Wakayama University

February 2026

概 要

ソフトウェア開発プロジェクトにおいて，コーディングスタイルを共通化するために，独自で設定したコーディング規約に違反するソースコードを自動検出する静的解析ツールが頻繁に使用される．しかし，静的解析ツールは規約違反コードを大量に検出するため，一部の規約違反コードのみが修正されることが多い．従来研究では，プロジェクトの修正履歴に基づき，機械学習を用いて修正すべき規約違反コードを推薦するモデルを開発しているが，修正履歴に正例数（修正された規約違反コード）が少ない場合に予測性能が低いという課題があった．本研究では，他のプロジェクトの修正履歴も学習データに含める違反修正要否判定モデルを提案する．提案手法として，全プロジェクトの修正履歴を混合する全学習手法と，規約違反の修正傾向が類似するプロジェクトの修正履歴のみを混合する選定学習手法の 2 種類を構築した．

170 のオープンソースプロジェクトを対象に検証を実施した結果，提案手法では，予測対象プロジェクトの修正履歴のみを学習する場合と比較して，正例数が 100 件から 500 件のプロジェクトにおいて適合率の向上が確認された．また，選定学習では正例数と修正率が高いプロジェクトで効果が大きい一方で，全学習ではプロジェクト特性に依存せず汎用的に効果が得られることが明らかになった．さらに，コーディング規約違反の種類ごとの分析では，複数プロジェクトで共通して発生している規約において，提案手法により多くの規約で正解率が向上することが確認された．特に，分析を行ったコーディング規約のうち，全学習が 9 種類の規約で，選定学習が 7 種類の規約で正解率の最大値を達成したのに対し，単一学習が最大値を取った規約は 3 種類のみであった．以上の結果から，複数プロジェクトの修正履歴を混合することで，コールドスタート問題を解消し，修正要否予測性能を向上させることが可能であることを示した．

本研究の貢献は，複数プロジェクトの履歴混合が，修正履歴の不均衡さを緩和し，判定精度を向上させることを定量的，定性的に実証した点にある．特に，開発現場の履歴に基づいた「現実的かつ文脈依存的な修正予測」の有効性を示したことは，静的解析ツールの実運用におけるノイズ削減と開発効率の向上に向けた重要な知見である．本研究が提示した手法は，リソースの限られた開発初期段階のプロジェクトにおいても高精度な品質管理を実現する補助となり，同分野の研究の進展に寄与するものである．

目 次

第 1 章　はじめに **1**

第 2 章　背景と関連研究 **3**

2.1　コード品質を担保するコーディング規約 3
2.2　静的解析ツール 3
2.3　静的解析ツール検出結果の自動分類 4
2.4　大規模言語モデルを利用した静的解析ツール検出結果の自動修正 5
2.5　Research Questions（RQs） 6

第 3 章　規約違反コードの修正要否判定方法とモデルの構築方法 **8**

3.1　モデル構築方法の概要 8
3.2　目的変数の計測 8
3.3　説明変数の定義 10
3.4　予測モデルの構築方法と特徴 11
3.4.1　単一学習モデル 11
3.4.2　全学習モデル 11
3.4.3　選定学習モデル（欠損値補完なし） 12
3.4.4　選定学習モデル（欠損値補完あり） 12
3.5　学習プロジェクトの選定方法 12
3.5.1　ジャッカード係数の測定 13
3.5.2　ユークリッド距離の測定 13
3.6　データセットの収集 14
3.6.1　対象言語とその選定理由 14
3.6.2　検証プロジェクトの選定方法 14

第 4 章　RQ1: 複数プロジェクトを混合する提案手法により修正予測性能は向上するか **18**

4.1　概要 18
4.2　分析方法 18
4.3　結果 18
4.3.1　選定学習手法による予測性能への影響 18
4.3.2　全学習手法による予測性能への影響 19
4.3.3　全学習と選定学習（欠損値補完あり）の予測結果の比較 20
4.3.4　データセット全体における予測結果 20
4.4　まとめ 21

第 5 章　RQ2: 正例が少ないプロジェクトであっても予測精度の改善に寄与する特徴量は存在するか **23**

5.1　概要 23
5.2　分析方法 23
5.3　分析結果 23

5.3.1 選定学習で予測性能が改善するプロジェクトの特徴 23
5.3.2 全学習で予測性能が改善するプロジェクトの特徴 25
5.4 まとめ 26

第 6 章 RQ3: 複数プロジェクトを混合する提案手法によりコーディング規約違反の種類ごとに予測性能は異なるか 28
6.1 概要 28
6.2 分析方法 28
6.3 結果 29
6.3.1 3 手法間のコーディング規約ごとの正解率の評価 29
6.3.2 正解率が変化するグループごとの分析 30
6.3.3 複数プロジェクトのデータを用いることで予測性能が向上するコーディング規約 31
6.4 まとめ 31

第 7 章 考察 33
7.1 概要 33
7.2 類似プロジェクトの集約による学習データの拡張と適合率向上 33
7.3 類似度測定方法の妥当性の評価 34
7.4 コーディング規約の種類による提案手法による予測結果への影響 36
7.5 提案手法は大規模言語モデルより優れた修正要否予測が可能か 37

第 8 章 妥当性への脅威 41
8.1 内的妥当性 41
8.2 外的妥当性 41

第 9 章 おわりに 42

第1章　はじめに

コーディング規約は，ソースコードの記述方法に関する禁止事項や推奨事項を定め, 可読性や保守性を高めるためのコーディングスタイルの取り決めである．ソフトウェア開発プロジェクトは，実装に参加する開発者にコーディングスタイルの共通化や, ソースコードの最適化のためにコーディング規約の遵守を促すことで, 可読性や保守性の高いソースコードを共有することができる [1]. コーディング規約は，プログラミング言語ごとに規約のテンプレートが存在し，開発者はテンプレートを参考にプロジェクトに適した規約の選定やカスタマイズを行っている．コーディング規約に従った開発は，開発者にとってのソースコードの理解容易性の向上や，バグの早期発見に貢献している [2] [3] [4].

コーディング規約に違反しているソースコード断片（以降，規約違反コード）を機械的に検出するために，多くのプロジェクトは静的解析ツールを使用している．静的解析ツールは，ソースコード中の規約違反コードを正規表現などにより，ソースコードを実行することなく検出できるため，継続的インテグレーションのプロセスの 1 つとして使用されることも多い．プログラミング言語別に提供される規約違反のリストでは，規約違反の種類が多数定義されているため，一部の違反のみ有効にしているプロジェクトも存在する [5]．また，プロジェクトで有効に設定した規約違反のうち，開発者は一部のみしか修正しておらず，従来研究の実証実験によれば 35%から 91%の規約違反コードは修正されていない [6]．しかし，静的解析ツールが検出する規約違反コードは膨大であり，その中から修正を要する規約違反コードを特定する必要がある．手動で判別することは開発効率の低下につながるため，修正が必要な規約違反コードを分類するための研究が行われている [7].

従来研究では，大量に検出される規約違反コードを，修正すべき違反とその他の違反に自動分類する手法が提案されている [8]．また，静的解析ツールの検出結果を修正すべき順番にランキング付けをする研究も数多く行われている [9]．両手法では，特定のプロジェクトにおいて，過去の規約違反コードの修正履歴を学習しモデルを構築する手法を提案している．当該手法は，学習データの総数が多い場合であっても，正例数の少ないプロジェクトで予測性能が低下する課題を有する．特定プロジェクトの修正履歴のみに依存する手法では，開発初期段階や小規模なプロジェクトにおいて，モデルの学習に必要な修正履歴が十分に確保できないデータ不足の問題（コールドスタート問題）が発生する．Tabassum らは，不具合予測やソースコード自動修正の分野で，データ不足やコールドスタート問題に対応するため，異なるプロジェクトの修正履歴を利用して学習データを補う手法を提案している [10]．規約違反コードの検出においても，異なるプロジェクトのデータを利用することでコールドスタート問題の解消が期待できる一方で，単に他プロジェクトのデータを混合するだけでは，実装方針やコーディングスタイルの差異がノイズとなり，期待に反して精度を低下させる懸念も存在する．

本研究では，プログラミング言語として Python を，静的解析ツールとして Flake8 を採用し，複数プロジェクトの修正履歴を予測モデルの学習に用いることによる規約違反コードの修正予測

性能を評価し，提案手法が規約違反コードの修正要否予測タスクのコールドスタート問題を解消するか否かを明らかにする．提案手法として，2 種類の予測モデルを構築する．1 つは，複数プロジェクトの修正履歴を単純に混合して学習する手法である（以降，全学習手法）．もう 1 つは，規約違反の修正傾向が類似する複数プロジェクトの修正履歴を混合して学習する手法である（以降，選定学習）．評価の結果，複数プロジェクト間で共通して発生しているコーディング規約違反の種類が多いプロジェクトにおいて，適合率の向上が確認された．

近年では大規模言語モデルによる自動修正も期待されるが，過剰修正の問題がある [11]．本研究のような修正要否判定は，大規模言語モデルが修正すべき対象を限定するための前処理としても重要である．

続く 2 章では, コーディング規約違反の検出と, 従来研究について述べる. 3 章では本研究における実験の全体像と提案手法について述べる．4，5，6 章では評価結果を示し, 7 章では考察を述べる. 8 章では本研究の妥当性への脅威について述べ, 9 章でまとめる.

第2章　背景と関連研究

2.1　コード品質を担保するコーディング規約

ソースコードの可読性や保守性といった品質を高水準で維持することは重要である．品質の高いソースコードを書くことで，ソースコードの理解の促進やバグの混入の予防など保守性を向上する効果が期待できる．また，品質の劣るソースコードには，良質なソースコードの約 15 倍の欠陥が含まれる [12].

大規模なソフトウェア開発では，ソースコードを実装した後に，実装者とは異なる開発者が，新規実装したソースコードにバグなどの問題が含まれていないかを確認するレビュー作業が発生する．可読性の高いソースコードはレビューコストの削減や技術的負債の抑制につながる [13]．つまり，可読性の高いソースコードを書くことは，ソフトウェア開発サイクル全体の促進につながる．

複数人によるソフトウェア開発においてソースコードの記述方法を共通化するために，ソフトウェア開発にコーディング規約を導入している．コーディング規約には，変数やクラスの命名規則，関数の長さや複雑度の上限といった可読性に関するルール，エラーの原因となる記述の検出，禁止事項，制限事項，推奨事項などが含まれる．コーディング規約は，プログラミング言語ごとに異なる内容や基準を持ち，多くの種類の規約が存在する．例えば，Python 言語には PEP8，Java 言語には Code Conventions for the Java Programming Language や Google Java Style Guide, JavaScript 言語には Google JavaScript Style Guide や Airbnb JavaScript Style Guide などが代表的な例として挙げられる．本研究で対象とする Python 言語の PEP8 は,「コードは書かれる回数よりも読まれる回数のほうが多い」という原理に基づいて定められたスタイルガイドである．本研究では，PEP8 に準拠したコーディング規約違反を検出する静的解析ツールである Flake8[1]を使用する．

開発者はコーディング規約を遵守したコーディングを行うことによって，一定のソースコードの品質を保つことができる．実装したコードをレビュアーが確認する際には，可読性が担保されたコードを読むことが可能となる．実装されたソースコードが，コーディング規約に従っているかどうかを開発者が手動で確認することには，多大な時間的コストを要する.．そのため，ソースコードのコーディング規約に従っていない箇所を自動的に特定するために静的解析ツールが用いられる．

2.2　静的解析ツール

静的解析ツールは，ソースコードがコーディング規約に従っているか否かの判定や，規約に違反している箇所の特定に用いられる．コーディング規約に従っていないソースコードを検出する

[1] https://flake8.pycqa.org/en/latest/

静的解析ツールも各プログラミング言語ごとに存在する．代表的な例としては，Python の Flake8 や JavaScript の ESLint，Java の Checkstyle が挙げられる．また，これらの静的解析ツールは規約の中から，違反として検出対象とする規約の種類を任意で決定することができる．検出する違反の種類を決定するためには，それぞれの静的解析ツールごとに定められたディレクトリに，決められたファイル名のテキストファイルを配置し，設定を記述することで制御できる．

ソフトウェア開発者は，各プロジェクトが設定する規約に調整を加えながら，静的解析ツールを用いて規約違反コードを検出している．プロジェクトの中には，継続的インテグレーション (CI) 環境に組み込むことで開発初期段階での不具合修正により，開発効率の向上を実現している [14] [15]．静的解析ツールは，ソースコード中の規約違反コードを網羅的に検出するが，その数は膨大であることが多く，開発者がそのすべてを確認，修正，保守するには多大なコストと労力を要する [16]．一部の違反を修正するにしても，優先的に修正すべき違反を区別するためには，経験やソースコードへの深い理解が求められる [17]．

本研究で使用する Python の静的解析ツールである，Flake8 について簡単に説明する．Flake8 は Python 言語を対象とした静的解析ツールであり，以下の 3 種類のツールを統合したラッパーである．

- PyFlakes: 論理エラーのチェック
- pycodestyle: PEP8 準拠のチェック
- McCabe: コードの複雑度（Cyclomatic Complexity）のチェック

Flake8 は実行時エラーとなるコード以外にも，可読性を低下させているコードについても検出可能である．また，McCabe では複雑度を測定することが可能であり，コードの複雑度は本研究で構築する機械学習モデルの説明変数にも利用する．これらのツール以外にも，プラグインとして様々なデバッグツールをカスタムで利用可能であるが，本研究では Flake8 の標準機能である，上記 3 種類のツールのみを利用する．

2.3 静的解析ツール検出結果の自動分類

従来研究では，静的解析ツールにより大量に検出した規約違反コードの中から，修正すべき規約違反コードを，機械学習を用いて分類する手法が提案されている [8] [18] [19]．Kim らは，静的解析ツールの出力をベイジアンネットワークに基づいて解析し，修正すべき規約違反コードを分類する手法を提案している [20]．また，検出された違反を修正すべきか否かの 2 クラスに分類するのではなく，検出された違反を修正すべき順にソートする，修正優先度付けを行う研究も数多く行われている [21] [22] [23]．これらの研究では，予測対象プロジェクトにおける規約違反の修正履歴を学習データとして用い，新たに検出された違反を評価対象とする機械学習モデル（規約違反コードの修正要否判定モデル）の構築と評価が行われている．

近年では大規模言語モデルを用いた，静的解析ツールの検出結果の分類手法が提案されている．Mohajer らは，大規模言語モデルである GPT-4 を利用し，静的解析ツールによる検出結果を，誤検出か否かを分類する手法を提案している [24]．また，静的解析ツールを用いずに，ソースコード自体を GPT に入力し，ソースコード中の警告の検出を行い，静的解析ツールとの検出精度を比

較している．結果として，大規模言語モデルは，従来手法より高い精度で静的解析ツールの検出結果の分類が可能であることが示された．また，静的解析ツールと大規模言語モデルでは，検出可能なコード品質低下の原因（違反の種類）が異なることが明らかにされている．

前述の従来研究を含め，多くの従来研究では，評価対象プロジェクトの修正履歴を学習データとして用いてモデルを構築している [25]．これは各プロジェクトでコーディングスタイルや開発体制が異なり，評価対象と同一プロジェクトの修正履歴を学習することで高い精度が得られることが示されている．しかし，一般的にコーディング規約の修正履歴は，修正されない負例が多い不均衡データであることが知られている [26]．不均衡データを学習に利用した結果，予測性能の低下を招く可能性がある．また，予測対象プロジェクトの違反修正履歴が少ない場合，学習データが小規模になり十分な学習ができない可能性がある．

従来研究では，名倉らが複数プロジェクトのデータを用いて規約違反の発生件数の増減を予測している [27]．一方で，個々の規約違反コードの修正有無の予測は対象としていないため，本研究とは異なるが，研究動機は類似している．異なるプロジェクトの修正履歴を用いることで，学習データの拡張による予測性能の向上が期待されるが，異なるコーディングスタイルを有するプロジェクトの修正履歴は予測性能の低下を招く．

そこで本研究では，評価対象プロジェクトと修正傾向が類似するプロジェクトを選定して混合することで，予測性能が低下するプロジェクトを排除し，予測性能が向上すると考えられる修正履歴を混合し学習データを拡張する．そして，拡張した学習データを学習に用いることにより，予測モデルの精度向上を目指す．

2.4 大規模言語モデルを利用した静的解析ツール検出結果の自動修正

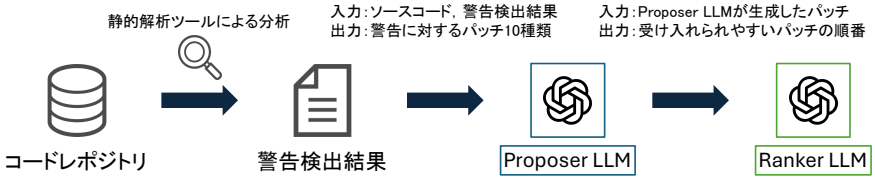

図 2.1: 大規模言語モデルを用いた静的解析ツール検出結果の自動修正手法の概略

近年の大規模言語モデルの発展に伴い，生成 AI を用いて静的解析ツールの解析結果からパッチを自動生成する研究が複数行われている [11] [28]．Wadhwa らは，プログラム中に存在する，コーディング規約に違反しているコードと静的解析ツールの結果から，修正パッチを自動生成をする研究を行っている [11]．図 2.1 に実験の大まかな流れを示す．提案されている手法では，パッチの自動生成において，二段階で大規模言語モデルを利用している．一つ目の大規模言語モデル（Proposer LLM）では，静的解析ツールによる検出箇所に対するパッチを複数生成する．二つ目の大規模言語モデル（Ranker LLM）では，生成されたパッチの中から，コンパイル可能かつ検出された問題が解消されているパッチから，開発者に受け入れられやすいパッチを順位付けする．

結果として，既存の深層学習を用いた自動パッチ生成の手法より高い精度でパッチの生成が可能であった．しかし，大規模言語モデルに修正パッチを生成させるタスクでは，プロンプトで静的解析ツールによる警告箇所のみを修正するように指示した場合であっても，関係のないコードまで修正を行ってしまうという欠点も報告されている．大規模言語モデルを用いて静的解析ツールの警告箇所の自動修正を行う研究において，パッチの生成対象となるコードは，静的解析ツールによって検出された全ての警告を対象としている．そのため，静的解析ツールの検出結果が修正を要さない箇所も修正しているため，修正すべきコーディング規約違反の判断は未だ人間が行う必要がある．提案手法は，静的解析ツールの検出結果を，修正を要する規約違反か否かを分類するため，自動パッチ生成を実行する前にパッチを生成する問題を絞り込むことで，レビューコストを小さくすることに貢献できる．

2.5 Research Questions（RQs）

本研究では静的解析ツールによって検出されたコーディング規約違反の修正要否を予測するにあたり，予測対象以外の複数プロジェクトの開発データを学習に使用することによる修正要否予測性能の向上を目的とする．本研究に関する先行研究は多く行われている．しかし，先行研究では予測対象のプロジェクトの過去の開発データのみを学習に使用しており，検証対象のプロジェクトには十分な修正履歴のあるプロジェクトを選択している．

そこで，本研究では予測対象以外の複数プロジェクトの学習データを機械学習モデルに学習させることによる，コーディング規約違反の修正要否予測の予測性能への影響を明らかにするために，以下の RQs を設定する．

- RQ1: 複数プロジェクトを混合する提案手法により修正予測性能は向上するか
- RQ2: 正例が少ないプロジェクトであっても予測精度の改善に寄与する特徴量は存在するか
- RQ3: 複数プロジェクトを混合する提案手法によりコーディング規約違反の種類ごとに予測性能は異なるか

RQ1 では，提案手法である，複数プロジェクトのデータを学習に使用することによる，コーディング規約違反の修正要否の予測性能への影響を明らかにする．提案手法ではプロジェクトのデータを単純に混合するのではなく，予測対象プロジェクトの修正履歴に混合した場合でも，ノイズにならないプロジェクトの選定を行い混合する．混合するプロジェクトの決定には，予測対象プロジェクトとコーディング規約違反に対する修正傾向が類似するプロジェクトを選定する．類似プロジェクトは，プロジェクト間の修正傾向のジャッカード係数とユークリッド距離を測定し決定する．そこで，予測対象プロジェクトと類似したプロジェクトを選定する方法の妥当性についても検証する．

RQ2 では，従来手法の課題として挙げた，予測対象プロジェクトの過去の開発データに正例が少ない場合に，提案手法によって予測性能が向上するのかについて分析を行う．また，予測性能が向上しやすい，つまり提案手法が有効に働くプロジェクトに特徴があるのかについて分析を行う．

RQ3 では，コーディング規約の種類ごとに予測結果の分析を行い，複数プロジェクトのデータを学習に使用するべき条件の分析を行う．具体的には，当該規約違反が共通して発生しているプロ

ジェクト数や，その中で修正傾向が類似しているプロジェクト数などの特徴に基づいて，どのようなコーディング規約の種類で複数プロジェクトのデータを学習に使用すべきかを明らかにする．

第3章　規約違反コードの修正要否判定方法とモデルの構築方法

3.1　モデル構築方法の概要

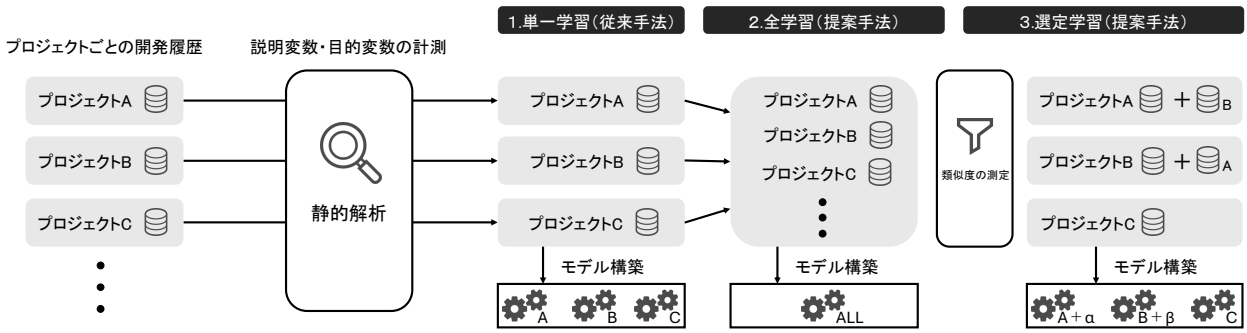

図 3.1: 提案手法の概略図

図 3.1 に，本研究で構築する予測モデルの構築方法の概略図を示す．本研究では，予測対象プロジェクトにおける規約違反の修正履歴に加え，他プロジェクトの修正履歴を混合することで，修正の必要性を高精度に判定するモデルを構築する．具体的には，学習データの決定方法の違いに基づき，以下の 4 種類のモデルを構築し，その予測性能を評価・比較する．なお，1 は従来研究に基づく比較手法，2 から 3 は本研究で提案する手法である．

1. 単一学習モデル（従来手法）
2. 全学習モデル（提案手法）
3. 選定学習モデル（提案手法）

これらの各モデルの具体的な構築手順および特徴の詳細については，3.4 節にて詳述する．

3.2　目的変数の計測

図 3.2 は，本研究で構築するモデルの説明変数および目的変数の計測位置を示す．目的変数は，分析期間内に検出した規約違反コードが，分析期間の最終リビジョンまでに修正される（正例）か否か（負例），言い換えると静的解析ツールにより規約違反として検出されるか否かと定義する．表 3.1 は，図 3.2 に示す 3 つのケースを例として，目的変数の分類を示す．ケース 1 は負例，ケース 2 およびケース 3 は正例とする．

具体的な目的変数の計測方法について図 3.2 を用いて説明する．

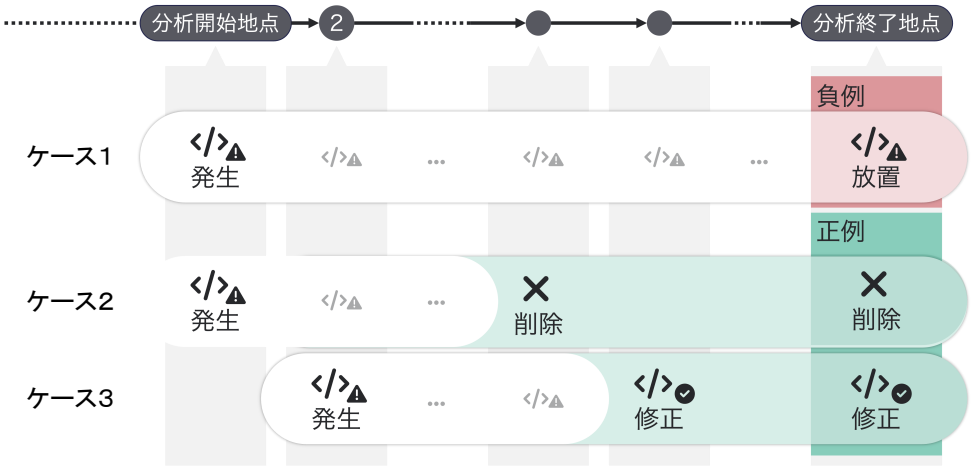

図 3.2: 説明変数と目的変数の計測位置

表 3.1: 目的変数の正例と負例の分類

分類	説明
負例（ケース 1）	規約違反がそのまま残存しているコード断片
正例（ケース 2）	違反コードが削除された断片
正例（ケース 3）	規約違反が修正されたコード断片

1. プロジェクトの修正履歴である GitHub リポジトリを取得する
2. 取得したリポジトリ内で，分析開始点までバージョンを戻す
3. 静的解析ツールをすべての Python ファイルに対して実行する
4. コーディング規約違反の場所と種類を記録する（図 3.2 のケース 1 とケース 2 の発生が記録される）
5. リポジトリを 1 つ次のコミットにチェックアウトする
6. 再度静的解析ツールを実行する
7. 新規コーディング規約違反があれば，追加記録する（図 3.2 のケース 3 の発生が記録される）
8. 以前記録したものが消えていれば，正例として記録する（図 3.2 のケース 2,3 が正例として記録される）
9. 分析終了地点まで 5 から 8 を繰り返し，残存している違反を負例とする

6 において，コードの移動があった場合，Git ではコードの削除と追加のように記録される．そこで本研究では，静的解析ツールによって検出されたコーディング規約違反行の前後行を比較し，完全一致するものがあれば同定し，コーディング規約違反を追跡することで，同じ違反に対して複数の修正履歴が計測されることを防ぐ．

3.3 説明変数の定義

本研究で使用する説明変数は，ソースコードから計測した特徴量 28 種類（行数，コメント行数，循環的複雑度など）を使用する．加えて，規約違反 ID を One-hot ベクトルとして加えたものや，Git による情報を説明変数とする．説明変数の一覧を表 3.2 に示す．使用する説明変数は，Wang らの研究において，静的解析ツールの誤検知分類タスクにおける重要な説明変数に関する研究が行われているため，その内容に基づき決定した [21]．多くの説明変数は，Wang らの従来研究に含まれているものを採用しているが，規約違反 ID を One-hot ベクトルとして加えている点が異なる．これは従来研究が，規約のカテゴリにのみ注視しており，規約の個々の種類まで対象としていなかったが，本研究では詳細な規約の種類まで分析を行うためである．本研究で使用した説明変数は，Java 言語を対象とした従来研究を参考にしているが，Python を対象とした従来研究においても同様の説明変数が使用されていることから，本研究の対象言語である Python においても妥当であると考えられる [19]．説明変数の計測タイミングは，規約違反が発生したリビジョンにおいて計測する．

従来研究では研究対象言語として，Java を主要な開発言語としているプロジェクトを利用している．本研究では研究対象言語に Python を採用しているため，従来研究で取得していたコードメトリクスの中で，対象言語の違いにより一部取得できないものがあったた．したがって，従来研究に基づいた説明変数ではあるが，完全一致はしていない．例えばインデントレベルは，Java では可読性を高めることが主要な目的であるが，Python においては制御構造を扱う重要な値となるため追加採用している．

コーディング規約違反の修正要否予測の評価方法

本研究では，コーディング規約違反の発生地点でのデータから，修正要否の予測を行う．各プロジェクトの規約違反修正履歴の評価用データを用い，機械学習モデルで予測を行い，その結果の評価を行う．評価には，適合率（式 3.1），再現率（式 3.2），F1 値（式 3.3），正解率（3.4）を利用する．本研究において適合率が高いことは，静的解析ツールが検出したコーディング規約違反のうち，修正の必要のない違反を削減できていることを意味する．また，再現率が高いことは，検出された違反のうち，修正すべき違反の網羅性が高いことを意味する．

各手法による予測結果における適合率，再現率，F1 値，正解率を比較することで，提案手法による静的解析ツールの検出結果の修正要否予測への有効性を明らかにする．

$$\text{適合率} = \frac{\text{修正が必要と予想できた違反数}}{\text{修正が必要と予想した違反数}} \tag{3.1}$$

$$\text{再現率} = \frac{\text{修正が必要と予想できた違反数}}{\text{修正された違反数}} \tag{3.2}$$

$$\text{F1 値} = \frac{2 \times \text{適合率} \times \text{再現率}}{\text{適合率} + \text{再現率}} \tag{3.3}$$

$$\text{正解率} = \frac{\text{正しく予測できた違反数}}{\text{全体の違反数}} \tag{3.4}$$

3.4 予測モデルの構築方法と特徴

本節では，コーディング規約違反に対する修正要否を予測するために構築する 4 種類の学習モデルの構築方法および特徴を詳述する．具体的には，学習データに利用するプロジェクトの決定方法に基づき,「単一学習モデル」,「全学習モデル」,「選定学習モデル（欠損値補完なし）」，および「選定学習モデル（欠損値補完あり）」の 4 手法を定義する．各モデルは，学習データの不足に起因するコールドスタート問題の解消と，プロジェクト固有の修正傾向の保持という，予測性能の向上における課題点と懸念点を考慮して設計している．

3.4.1 単一学習モデル

本節で述べる単一学習モデルとは，予測対象となるプロジェクトの過去の修正履歴のみを学習データとして用いるモデルを指す [8]．図 3.1 の単一学習に示すように，本モデルは従来研究の手法と同様，予測対象プロジェクトの過去の開発データのみを学習に利用する．具体的には，プロジェクト A の修正要否を予測する場合，プロジェクト A 自身の過去の修正履歴のみを学習に用いる．したがって，予測モデルはプロジェクトごとに個別に構築される．

ここで，従来手法と本研究の差異について述べる．従来手法は，セキュリティバグや致命的なエラーを引き起こすバグに関する静的解析ツールの警告のみを対象としていた．これに対し，本研究では，実行時エラーには直結しないコーディングスタイルに関する警告まで予測対象を拡張している．

単一学習モデルは，当該プロジェクトの過去の修正履歴のみを用いるため，プロジェクト固有のコーディングスタイルを捉えたモデルの構築が可能である．その一方で，プロジェクトの開発初期段階などの要因により蓄積された修正履歴が不十分な場合には，予測モデルの学習が十分に行えないコールドスタート問題が存在する．

3.4.2 全学習モデル

全学習モデルとは，本研究の評価対象とする全てのプロジェクトの修正履歴を学習データとして用いるモデルを指す．図 3.1 の全学習に示すように本モデルは，特定のプロジェクトに限定せず，本研究で分析対象とする全プロジェクトの修正履歴を混合して学習に使用する予測モデルである．したがって，予測対象となるプロジェクトに関わらず，同一の予測モデルを用いて修正要否を予測する．具体的には，プロジェクト A およびプロジェクト B のいずれを予測対象とする場合であっても，修正要否の判定に用いられる予測モデルは同一のものとなる．

本モデルは，学習に大量の修正履歴を用いることが可能であるため，前述のコールドスタート問題に対して頑健であるという利点を持つ．しかし，プロジェクトごとの固有のコーディングスタイルや修正傾向は考慮されない．例えば，プロジェクト A では修正傾向にある規約が，プロジェクト B では修正されない傾向にある場合，これらのデータが混在することで互いのノイズとなる

懸念がある．プロジェクト間で修正傾向が異なる規約が学習データに含まれる場合，予測対象プロジェクトにおいて十分な修正履歴があるプロジェクトでは，予測性能の低下を招く恐れがある．

3.4.3　選定学習モデル（欠損値補完なし）

選定学習モデル（欠損値補完なし）とは，予測対象プロジェクトの修正履歴に加え，当該プロジェクトと発生している規約違反の種類と，規約違反に対する修正率が類似するプロジェクトを選定し，それらの修正履歴を混合して学習データとして用いるモデルを指す．本モデルでは，プロジェクト間での類似度算出に基づき学習データを選定するが，その際の欠損値の扱いには特別な処理を要する．具体的には，比較するプロジェクト間で一方のみが発生している規約違反が存在する場合，他方のプロジェクトにおける当該規約の修正率は欠損値となる．本モデルでは，このような欠損値を含む規約 ID については類似度算出の対象から除外するものとする（選定方法の詳細は 3.5 節参照）．

本モデルは，予測対象プロジェクトと類似度の高いプロジェクトのみを選定して学習データに加えるため，全学習モデルの懸念点である「修正傾向の異なるプロジェクトの混在によるノイズ」を抑制しつつ，学習データを拡充することが可能である．

3.4.4　選定学習モデル（欠損値補完あり）

選定学習モデル（欠損値補完あり）とは，前述の「欠損値補完なし」のモデルと同様に，予測対象プロジェクトの修正履歴と，選定した類似プロジェクトの修正履歴を組み合わせて学習データとするモデルを指す．本モデルの特徴は，プロジェクト間で規約違反の発生状況が異なるために生じる欠損値を補完し，類似度を算出した上で学習に利用する点にある．具体的には，ある規約の修正率が欠損している場合，当該プロジェクトにおける他の規約の平均修正率を算出し，その値で欠損値を補完する（選定方法の詳細は 3.5 節参照）．

本モデルでは，欠損値補完を行うことにより，欠損値補完を行わない選定学習と比較して，より多くの規約違反の種類を用いた類似度算出が可能となる．これにより，欠損値補完を行わない選定学習より，修正傾向の異なるプロジェクトの混合を抑制できるという利点がある．その一方で，欠損値補完を行わない選定学習と異なり，修正率の距離の計算時に計算から null を除外しなくなり，類似度の判定基準が厳格化されるため，学習データとして採用できる類似プロジェクトが減少する可能性がある．

3.5　学習プロジェクトの選定方法

全学習モデルおよび 2 種の選定学習モデルでは，予測対象プロジェクトの修正履歴に加え，他プロジェクトの修正履歴を学習データに含めることで正例データを拡張する．全学習モデルではデータセット内の全プロジェクトを対象とするのに対し，選定学習モデルでは，予測対象プロジェクトと規約の設定状況および修正率が類似するプロジェクトの修正履歴のみを選択的に利用する．

プロジェクト選定にあたっては，各プロジェクトの規約設定状況と規約の種類ごとの修正率を計測し，類似度を算出する．各規約違反における修正率は，式 (3.5) によって定義する．

$$修正率 = \frac{同一種類の違反の修正された数}{同一種類の違反を検出した数} \quad (0 \leq 修正率 \leq 1) \tag{3.5}$$

本研究において，プロジェクトの選定基準として「設定された規約」と「修正率」の類似性を採用した動機を以下に述べる．まず，規約の設定状況の類似性を考慮することで，実装方針が類似するプロジェクトを抽出可能である．これにより，自プロジェクトとは無関係な規約に基づくノイズの混合を抑制できる．次に，修正率の類似性を考慮することで，規約違反に対する修正の方針が類似するプロジェクトを選定できる．プロジェクト間での修正傾向の不一致は，モデルの学習において互いにノイズとなり，予測性能の低下を招く要因となる．したがって，これら 2 つの観点から類似プロジェクトの選定を行う．

3.5.1 ジャッカード係数の測定

各プロジェクトが設定する規約の類似度は，本研究で対象とする Flake8 で検出可能な規約違反を対象とする．プロジェクトの規約設定有無（バイナリーデータ）の類似度を判定するためには，ジャッカード係数を用いる．表 3.3 に，プロジェクトごとの各規約違反に対する修正率の例を示す．表中の「規約 1」は，プロジェクト A では規約 1 が 0.70 の確率で修正されており，プロジェクト B では 0.60 の割合で修正されている．そして，プロジェクト C では規約 1 への違反が一度も発生していないため欠損値となる．プロジェクト A とプロジェクト B のジャッカード係数を計算する場合，発生しているコーディング規約違反のうち，規約 2 以外が共通して発生しているため，ジャッカード係数は式 3.6 のように求められる．

$$Jac(A, B) = \frac{3}{4} = 0.75 \tag{3.6}$$

3.5.2 ユークリッド距離の測定

プロジェクト間の規約違反に対する修正率（スカラーデータ）の類似度の測定には，ユークリッド距離を用いる．なお，規約の修正率は，一方のプロジェクトのみが規約を設定している場合，他方のプロジェクトでは欠損値となるため，次の 2 通りの方法で欠損値補完し，類似度を計測する．

- 欠損値を計算から除外してユークリッド距離を測定する方法（選定学習モデル（欠損値補完なし））
- 欠損値を当該プロジェクトの平均修正率で補完し，ユークリッド距離を測定する方法（選定学習モデル（欠損値補完あり））

具体的なユークリッド距離の測定の説明を行うために，表 3.3 中のプロジェクト A とプロジェクト B のユークリッド距離の測定を例に挙げる．表 3.3 の各値が修正率を示しているため，選定学習手法（欠損値補完なし）の修正率のユークリッド距離は，式 3.7 のように求められる．式中の平方根における第 1 項は規約 1 の修正率の差（距離）を示している．規約 2 はプロジェクト B では違反が発生していないため，計算から除外している．続く 2 項および 3 項は，それぞれ規約 3，規約 4 に対する修正率の差を示している．

$$Euc(A,B) = \sqrt{(0.70-0.60)^2 + (0.30-0.20)^2 + (0.20-0.30)^2} \tag{3.7}$$

最後に，欠損値補完を行う場合のユークリッド距離を算出する方法について説明する．プロジェクト B では，規約 2 が発生していないため，プロジェクト B の他規約の修正率の平均値で補完して計算を行う．その結果，選定学習手法（欠損値補完あり）の類似プロジェクト分析に利用するユークリッド距離は式 3.8 のように求められる．式 3.8 では，式 3.7 で示した 3 つの項に加え，新たに第 2 項として欠損値補完された規約 2 の修正率（0.40）に基づく差の計算が追加されている．このように欠損値補完を用いることで，発生していない規約に対しても距離の計算を行うことが可能である．

$$Euc'(A,B) = \sqrt{(0.70-0.60)^2 + (0.25-\mathbf{0.40})^2 + (0.40-0.30)^2 + (0.20-0.30)^2} \tag{3.8}$$

ここで欠損値を平均値で補完しているのは，特定のプロジェクトにおいて発生していない規約は，そのプロジェクトの開発者が設定ファイルで当該規約を無効にしているか，単に出現していないだけかの区別が困難なため，プロジェクト全体の平均的な修正傾向を代入することで近似するためである．区別が困難な理由は，分析期間内で設定ファイルの変更が行われるためである．

実験を行う際には，表 3.3 のような修正率の表を全プロジェクトの全コーディング規約に拡張したものを計算し，類似度の計算に利用する．プロジェクト間でそれぞれの指標が類似するか否かは，実証データに基づき閾値を決定する．具体的な閾値の決定方法は，ジャッカード係数（0.0 から 1.0 をステップ 0.1）とユークリッド距離（1.0 から 5.0 をステップ 1.0）のすべての組み合わせ（50 種類）に対してグリッドサーチを行い，学習データにおいて最も高い予測性能のものを採用する．

3.6 データセットの収集

3.6.1 対象言語とその選定理由

検証に使用するプログラミング言語には，Python を採用する．Python は，近年の機械学習やデータ分析における需要の高まりを背景に，言語としての需要が増している．また，Python 言語の静的解析ツールである Pylint を対象とした，静的解析ツールの検出結果を機械学習により分類する同様の研究も行われている [19]．本研究においても Python を対象とすることで，従来手法との比較や検証が可能となることから，Python を採用する．

3.6.2 検証プロジェクトの選定方法

検証に使用するデータセットとして，OSS ライブラリ検索サービスである Libraries.io[1]に登録されている Python ライブラリを用いる．検証に使用するライブラリの条件としては，以下の条件を満たすものを採用した．

[1] `https://libraries.io/`

- SourceRank 上位 2,000 プロジェクト
- GitHub でソースコードが公開
- 静的解析ツールである Flake8 の設定ファイルを保有
- 学習データ，評価データの双方に正例・負例が存在
- 規約違反データが 500 件以上，20,000 件以下
- 開発リポジトリが重複しているプロジェクトを削除

上記の条件を満たした 170 プロジェクトのデータを検証に使用する．コーディング規約違反のデータを取得する期間は，2024 年 1 月 1 日から 2024 年 12 月 31 日までの 1 年間のコミット履歴を対象とする．

本研究において，対象とする規約違反件数を 500 件以上かつ 20,000 件以下に制限した理由は，以下の 2 点である．

第一に，統計的な評価の信頼性を担保するためである．データセットを学習用とテスト用に分割した際，テスト用データが不足していると，少数のデータが評価指標に過度な影響を及ぼし，結果の安定性が損なわれる懸念がある．本研究では，データを 4 対 1 の比率で分割することを前提とし，テスト用データとして最低限 100 件を確保するため，全データ数が 500 件以上であることを条件とした．

第二に，静的解析ツールが実質的に運用されていないプロジェクトを排除するためである．規約違反数が極端に多いプロジェクトは，設定ファイルのみを保持し，実際の開発プロセスではツールのコーディング規約違反を無視している可能性が高い．図 3.3 に示す適用前のプロジェクト数分布を確認すると，違反数 20,000 件付近を境に分布の不連続性が確認できる．そのため，解析対象としての妥当性を考慮し，区切りの良い数値である 20,000 件を上限として設定した．

取得したデータセットの規約違反を図 3.4 に示す．規約違反の発生量としては，中央値が 2,160 件で，500 件から 1,000 件のプロジェクトが最も多い．また正例数と負例数の割合が，中央値で正例数が 0.34 という負例数が多い不均衡データであった．

表 3.2: 変数一覧と説明

種類	変数名	説明
flake8 情報	Violation ID	静的解析ツールが検出した違反の種類を示す ID（One-hot）
	Category	違反のカテゴリ（One-hot）
	Violation Line Number	違反が検出された行番号
Git 情報	File Change Frequency	ファイルの変更頻度
	Lines Added Past 25 Revisions	過去 25 リビジョンで追加された行数
	Lines Added Past 3 Months	過去 3ヶ月で追加された行数
コード特徴量	File Size	ファイルサイズ（バイト数）
	Total Lines	ファイルの総行数
	Code Lines	コード行数
	Comment Lines	コメント行数
	Blank Lines	空白行数
	File Depth	ファイルの階層の深さ
	Filename Length	ファイル名の長さ
	Total Functions	ファイル内の関数数
	Total Classes	ファイル内のクラス数
	Total Imports	インポート文の数
	Total Variables	変数の数
	Cyclomatic Complexity	循環的複雑度（ファイルレベル）
	Line Length	違反行の長さ（文字数）
	Line Length No Whitespace	違反行の長さ（空白を除く）
	Indent Level	違反行のインデントレベル
	Line Complexity	違反行の複雑度
	Special Chars	違反行の特殊文字数
	Variable Count	違反行の変数数
	Function Calls	違反行の関数呼び出し数
	Operators	違反行の演算子数
	In Function	違反が関数内にあるかどうか（0/1）
	Function Params	関数のパラメータ数
	Function Lines	関数の行数
	Function Complexity	関数の複雑度
	In Class	違反がクラス内にあるかどうか（0/1）
	Class Methods	クラスのメソッド数
	Class Lines	クラスの行数

表 3.3: プロジェクト別規約遵守状況

	規約 1	規約 2	規約 3	規約 4
プロジェクト A	0.70	0.25	0.40	0.20
プロジェクト B	0.60	-	0.30	0.30
プロジェクト C	-	0.70	-	0.60

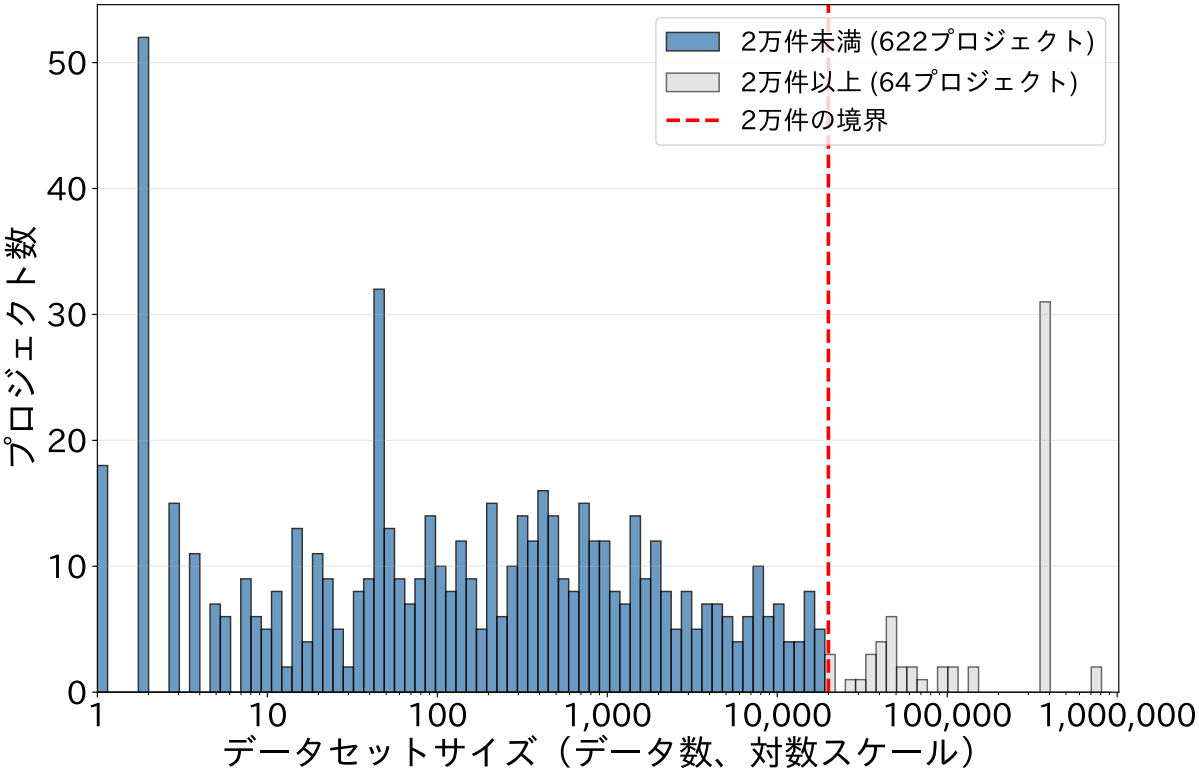

図 3.3: フィルタリング前のプロジェクトごとの規約違反数分布

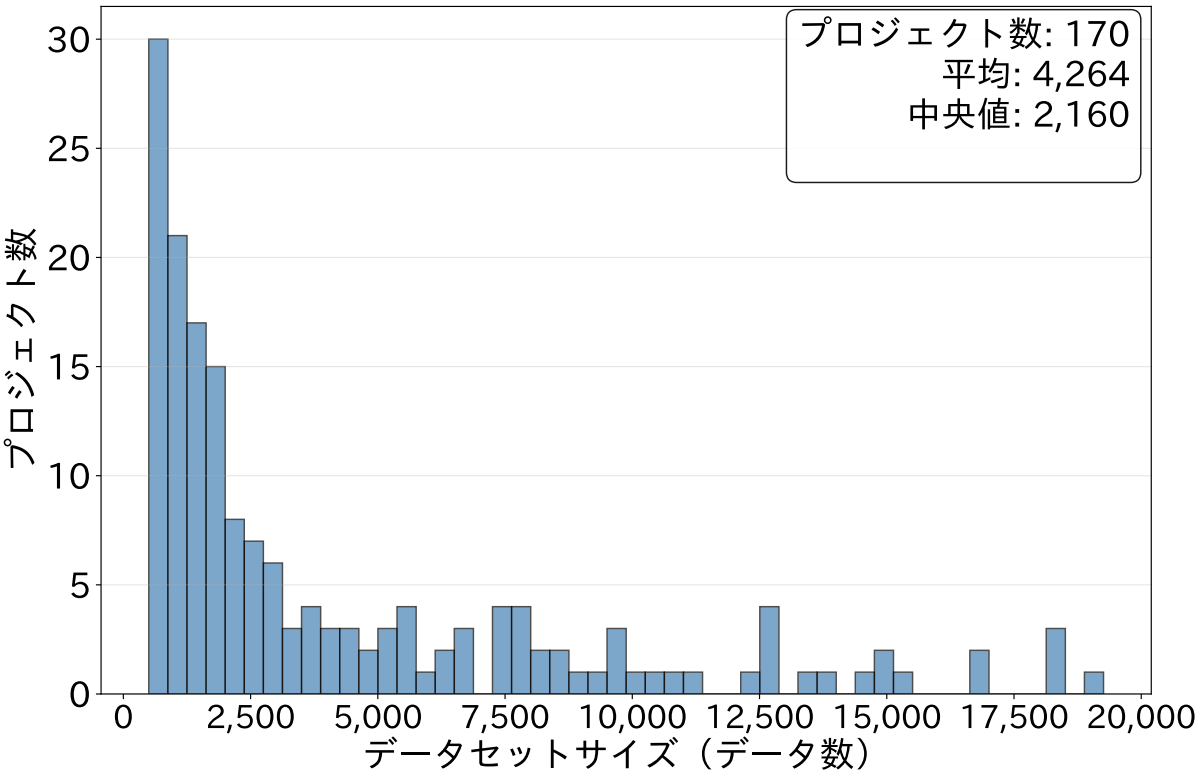

図 3.4: フィルタ条件適用後のプロジェクトごとの規約違反数分布

第4章　RQ1: 複数プロジェクトを混合する提案手法により修正予測性能は向上するか

4.1　概要

RQ1 では従来手法である単一学習と提案手法である全学習，選定学習（欠損値補完なし），選定学習（欠損値補完あり）との修正要否の予測結果を比較する．具体的には，提案手法によって，従来手法の課題点としてあげていた，学習データの正例数が少ない場合において予測性能が向上するか否かを明らかにする．また，提案手法を学習データの正例数で限定せずに全てのプロジェクトに適応した結果についても分析する．4.3 節では，選定学習手法の評価を行う際に，主に選定学習手法（欠損値補完あり）について言及し，その理由について 7.3 節にて詳説する．

4.2　分析方法

RQ1 の分析では，単一学習と提案手法（全学習，選定学習（欠損値補完なし），選定学習（欠損値補完あり））の 4 手法による修正要否予測の結果を比較する．評価指標として，適合率，再現率，F1 値，正解率を用いる．分析は，学習データの正例数に基づいて 4 つの区間に分類し，各手法の予測性能の改善度を箱ひげ図により可視化する．また，データセット全体における予測結果についても，箱ひげ図により各手法の性能を比較し，ウィルコクソンの符号順位検定（有意水準 0.05）により統計的有意差を検証する．

4.3　結果

4.3.1　選定学習手法による予測性能への影響

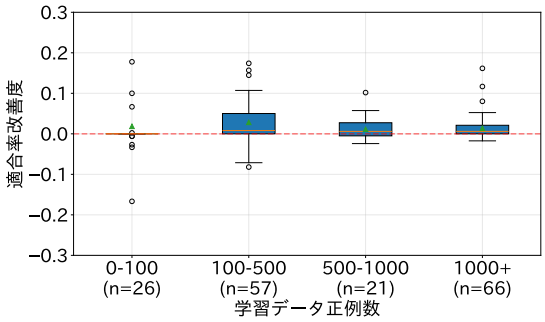

図 4.1: 選定学習（欠損値補完あり）による適合率の改善度

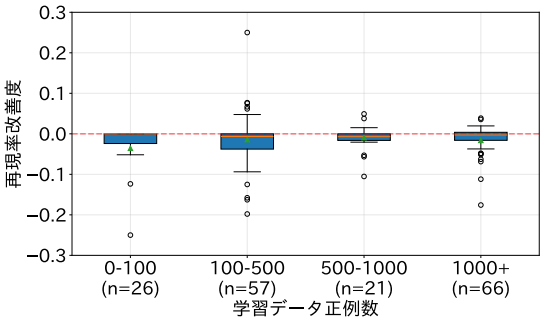

図 4.2: 選定学習（欠損値補完あり）による再現率の改善度

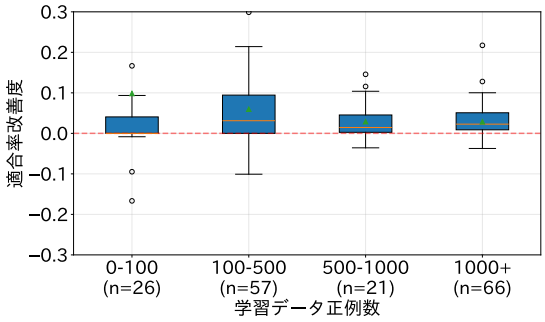

図 4.3: 選定学習による適合率の改善度

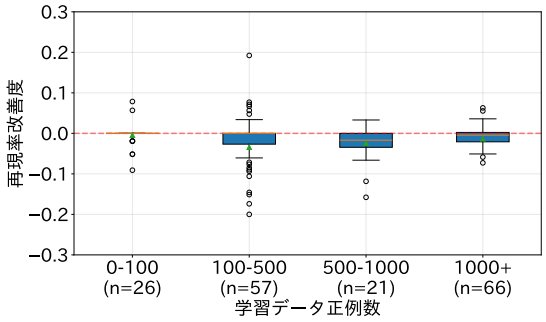

図 4.4: 選定学習による再現率の改善度

提案手法の導入による影響を，学習データの正例数に基づいて分析した結果を示す．図 4.1 は，提案手法（欠損値補完あり）による単一学習からの適合率の変化量を示した箱ひげ図である．また，図 4.2 には，同様の分析を再現率の観点で行った結果を示す．いずれの図も横軸は，学習データに含まれる正例数に基づき 4 つの区間に分類している．

これら 2 つの図から，提案手法（欠損値補完あり）の導入によって，適合率が上昇する一方で再現率が低下する傾向が確認できる．特に，正例数が 100 件から 500 件の範囲において，箱ひげ図の分散が最も大きくなっている．複数プロジェクトの修正履歴を学習する提案手法は，モデル構築の前に学習データの混合を行って学習を行っている．そのため，予測対象プロジェクト内の学習データ数が少ない場合に混合したデータの重要度が高まり，予測性能の変動が大きくなったと考えられる．このことから，正例数が 100 件から 500 件のプロジェクトにおいて，提案手法が与える影響が特に大きいと言える．図 4.3 と図 4.4 に，欠損値補完を用いない選定学習の，同様の分析結果を示す．結果から，欠損値補完を用いない選定学習においても，選定学習（欠損値補完あり）と同様の傾向が見られた．

4.3.2 全学習手法による予測性能への影響

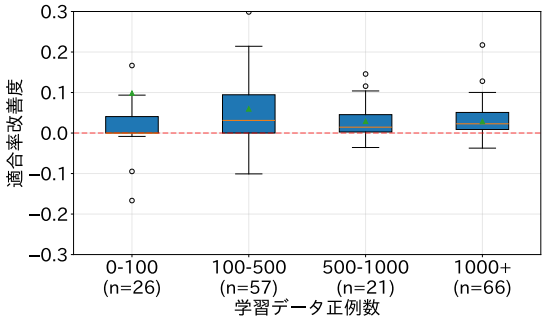

図 4.5: 全学習手法による適合率の改善度

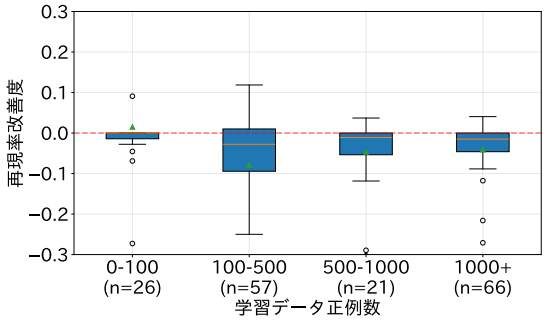

図 4.6: 全学習手法による再現率の改善度

次に全学習手法による，修正予測性能への影響について述べる．全学習手法での学習データの正例数ごとの適合率の改善度を図 4.5 に示し，再現率の改善度を図 4.6 に示す．全学習手法の結果からも，選定学習の際と同様に単一学習の結果と比較して適合率が上昇傾向にあり，再現率が低下傾向にあることがわかる．

4.3.3　全学習と選定学習（欠損値補完あり）の予測結果の比較

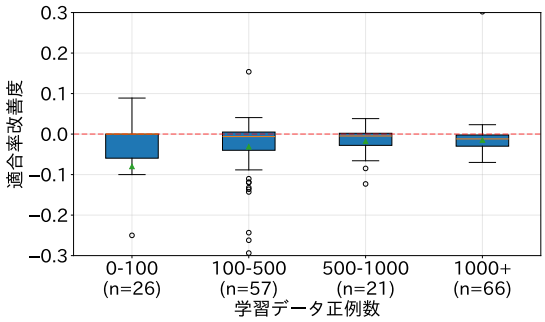

図 4.7: 全学習に対する選定学習（欠損値補完あり）の適合率改善度

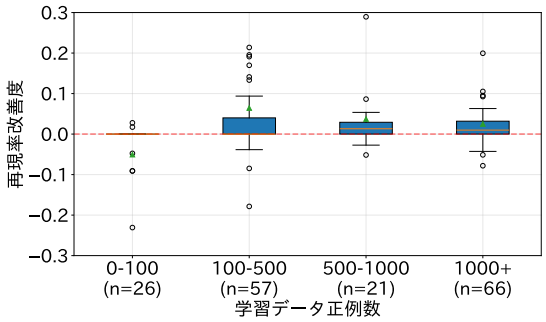

図 4.8: 全学習に対する選定学習（欠損値補完あり）の再現率改善度

提案手法である，全学習と選定学習（欠損値補完あり）間の修正要否予測結果の比較を行う．図 4.7 に，全学習の適合率から，選定学習による適合率の改善度を示す．図 4.8 に，全学習の再現率から，選定学習による再現率の改善度を示す．図から見てとれるように，全学習と選定学習（欠損値補完あり）を比較した場合，全学習の方が適合率の高いプロジェクトが多く，選定学習の方が再現率の高いプロジェクトが多い結果となっている．

ここまでの結果から，全学習と選定学習（欠損値補完あり）の比較において，以下の知見が得られた．まず，箱ひげ図の分散に違いはあるものの，改善度合いの分布に大きな違いは見られない．次に，単一学習と比較した場合，両手法ともに適合率が改善するプロジェクトと再現率が低下するプロジェクトが共通している．さらに，全学習は選定学習（欠損値補完あり）と比較して，適合率の改善幅と再現率の低下幅が大きい傾向にある．全学習は，大規模なデータ量によって学習データを拡張できる一方で，ノイズとなってしまうデータも大量に学習する．特に，本研究が対象とするコーディング規約違反の修正履歴は負例数と比較して正例数の方が少ない不均衡データである．本研究で取得したプロジェクト別の修正率の中央値は 0.34 であり，負例数が正例数の 3 倍程度存在するプロジェクトが半数を占める．そのため，全学習では，負例数が多い修正履歴データを学習することで，判定が厳しくなり，再現率が低下したと考えられる．

4.3.4　データセット全体における予測結果

これまでの分析では，学習データの正例数に基づいて提案手法の効果を検証した結果，特に正例数が 100 件から 500 件の範囲において提案手法が与える影響が大きいことを確認した．しかし，データセット内のプロジェクト全体の規約違反の修正要否予測の結果を図 4.9 に示すように，全体として見ると提案手法による改善は限定的であった．図は左から各手法の予測結果の適合率，再現率，F1 値，正解率を順に示し，縦軸にその値を示している．選定学習と選定学習（欠損値補完あり）の 2 手法では，プロジェクト間類似度の測定の際に，ジャッカード係数とユークリッド距離のそれぞれに閾値を設けている．閾値は，ジャッカード係数とユークリッド距離の組み合わせの全 50 通りの中から，学習データでの予測性能が最も高い値を採用した場合の，予測結果を使用している．

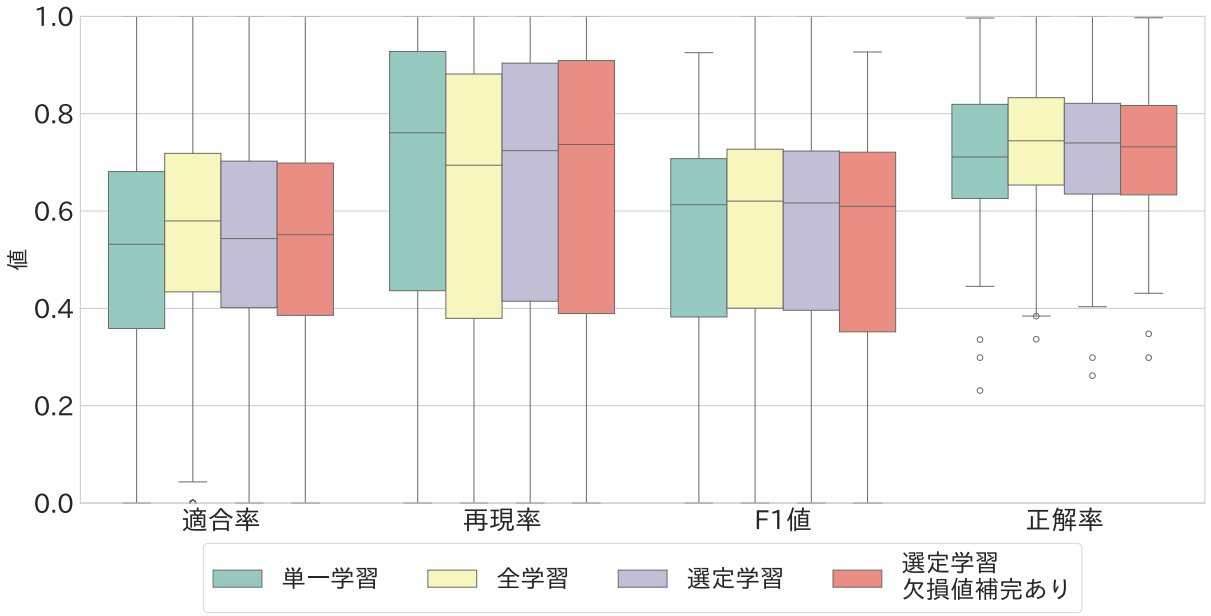

図 4.9: 4 手法による予測結果の箱ひげ図

結果としては，単一学習の結果と比較して提案手法 3 種類では，適合率と正解率が僅かに改善し，再現率が僅かに低下する結果となった．各メトリクスに対して，4 手法間の全ての組み合わせでウィルコクソンの符号順位検定（有意水準 0.05）を行ったところ，全ての組み合わせにおいて，統計的に有意な差はなかった．

データセット全体のプロジェクトを対象とした検証において，予測性能に顕著な差が見られなかったのは 2 章でも懸念していた通り，本提案手法が本来，学習データの確保が困難なプロジェクトに対する精度向上を主眼として設計したものであることに起因すると考えられる．また，本研究で使用したデータセットは，ライブラリ検索サービス上で評価の高いライブラリを採用しているため，コーディング規約違反の修正予測を行う学習に必要な量の修正履歴を有するプロジェクトが多い．そのため，データセットの大部分を占める正例数の多いプロジェクトでは，単一学習ですでに精度が飽和しており，提案手法の利点が小さく，全体的な結果として有意差が出なかったと考えられる．

4.4 まとめ

本章では，コーディング規約違反の修正要否予測における提案手法の有効性を検証した．得られた知見を以下に総括する．

本研究で提案している，予測対象プロジェクト以外のデータを学習に使用する手法が，有効に働くプロジェクトを調査したところ，学習データ内の正例数が 100 件から 500 件のプロジェクトにおいて，提案手法による精度の変化の影響が最も大きくなることを確認した．特に，正例数が少ないプロジェクトにおいて，提案手法の導入によって適合率が上昇する一方で再現率が低下する傾向が確認できた．

全学習手法と選定学習（欠損値補完あり）の予測結果の比較では，全学習は選定学習（欠損値補完あり）と比較して，適合率の改善幅と再現率の低下幅が大きいことが明らかになった．全学

習と選定学習の両手法で，予測性能の変化傾向が共通していることから，単一学習では捉えきれない修正要否を他プロジェクトのデータから補完できている．

一方で，データセット全体のコーディング規約違反の修正予測において，従来手法である単一学習と比較して，提案手法である全学習と選定学習（欠損値補完あり）では，適合率の僅かな改善と，再現率の僅かな低下が見られたものの，比較した 4 手法間では，適合率，再現率，F1 値に統計的有意差は見られなかった．これは，本研究の対象とするプロジェクトがデータが十分に収集できないプロジェクトに焦点を当てている一方で，本研究で使用したデータセットは，ライブラリ検索サービス上で評価の高いライブラリを採用しているため，コーディング規約違反の修正予測を行う学習に必要な量の修正履歴を有するプロジェクトが多く，データセットの大部分を占める正例数の多いプロジェクトでは，単一学習ですでに精度が飽和しており，提案手法の利点が小さかったためであると考えられる．

以上の結果から，修正履歴が十分に収集できないプロジェクトにおいて，他プロジェクトのデータを活用した学習手法が修正要否予測に与える影響について，以下の考察が得られる．まず，正例数が 100 件から 500 件のプロジェクトにおいて提案手法の効果が最も大きくなるという知見は，データ不足による予測性能の低下が顕著に現れる範囲と，他プロジェクトデータによる補完効果が有効に働く範囲の境界を示唆している．この範囲を超えるプロジェクトでは，単一学習でも十分なデータ量が確保されているため，他プロジェクトデータの追加による改善幅は限定的であると考えられる．一方，この範囲を下回るプロジェクトでは，データ量が極めて少ないため，他プロジェクトの修正履歴の追加だけでは予測性能の改善が困難であることが示唆される．そのため，開発初期では，他プロジェクトのデータを全て学習した汎用的なモデルである全学習モデルを利用し，正例数が 100 件以上になった段階で，選定学習モデルを利用することが推奨される．具体的には，学習データの正例数が 500 件未満のプロジェクトにおいて，提案手法によって平均 228 個の静的解析ツールの検出結果が趨勢不要であることを正確に予測可能である．そして，プロジェクトの正例数が 500 件を超えた段階では，予測対象プロジェクトのみの修正履歴を学習に用いる単一学習モデルを利用することが推奨される．

第5章　RQ2: 正例が少ないプロジェクトであっても予測精度の改善に寄与する特徴量は存在するか

5.1　概要

RQ1 では，従来手法である単一学習と提案手法との予測結果の比較を行った．その結果，複数プロジェクトのコーディング規約違反の修正履歴を混合することで，予測性能が向上するプロジェクトの条件が明らかになった．そこで，RQ2 では，提案手法によって予測性能が改善したプロジェクトと低下したプロジェクトに分割し，それぞれの特徴について分析することで，提案手法が有効に働くプロジェクト特性を明らかにする．また，提案手法によって予測性能が低下したプロジェクトについても分析し，提案手法が要因となり予測性能が低下しているか否かを明らかにする．

5.2　分析方法

単一学習の適合率と提案手法の適合率の差が大きい，学習データの正例数が 100 件から 500 件のプロジェクトのうち，選定学習によって予測性能が改善した群，低下した群に分割し，説明変数と混合するプロジェクトの特徴を分析する．単一学習と比較する選定学習は，欠損値補完を用いる選定学習を利用する．また，選定学習によって予測性能が改善したか否かについては，図 4.1 と図 4.2 で単一学習から適合率，再現率が向上したプロジェクトと，低下したプロジェクトで比較する．

分析する内容として，学習データ内に存在するコーディング規約違反の種類の傾向や説明変数をはじめとする，コードメトリクスや学習データ数，正例数，負例数のような数値的なデータの特徴の傾向を分析する．分析には，マンホイットニーの U 検定（有意水準 0.05）による統計的有意差の確認と，クリフのデルタによる効果量を明らかにする．分析対象である予測結果の改善群と低下群の分布は正規分布ではないため，ノンパラメトリック検定であるマンホイットニーの U 検定を採用した．クリフのデルタについては，一般的に用いられる基準に基づき評価を行う [29].

5.3　分析結果

5.3.1　選定学習で予測性能が改善するプロジェクトの特徴

適合率が上昇している要因を特定するため，提案手法による適合率の変化幅が最も大きかった，正例数が 100 件から 500 の範囲にある 57 プロジェクトに対象を絞り，出現する規約違反 ID の種類に着目した詳細な分析を行った．学習データの正例数が 100 件から 500 のプロジェクトで，選

表 5.1: 適合率が改善したプロジェクト（37 件）で出現する規約違反 ID（上位 10 件）

Violation ID	出現回数	出現プロジェクト数
E501	31,124	35
F401	1,054	28
E203	357	22
E402	525	15
F841	270	15
F811	33	15
E302	263	12
F821	127	11
E722	127	11
E303	100	11

表 5.2: 適合率が低下したプロジェクト（20 件）で出現する規約違反 ID（上位 10 件）

Violation ID	出現回数	出現プロジェクト数
E501	20,509	20
F401	1,355	16
E203	1,284	11
F841	380	10
E231	305	9
E265	587	9
E302	490	9
E721	180	9
E128	1,487	8
E303	420	8

定学習（欠損値補完あり）によって単一学習から適合率が改善したプロジェクトは，向上が 37 プロジェクト，低下が 20 プロジェクトであった．

まず，適合率が改善したプロジェクト群において最も頻繁に出現する規約は ‘E501’（1 行あたりの文字数制限）であった．しかし，表 5.1 および表 5.2 の比較から明らかな通り，本規約は適合率が低下したプロジェクト群においても同様に高い頻度で出現している．‘E501’ はソースコードの書式に関する一般的な違反であり，特定のプロジェクト群に固有の性質を示す特徴とは言い難い．‘F401’，‘E203’，‘F841’，‘E302’，‘E303’ も同様に適合率の向上したプロジェクトと低下したプロジェクトの両群において頻繁に出現している．

適合率の向上しているプロジェクトのみ出現するコーディング規約の種類として，‘E402’，‘F811’，‘F821’，‘E722’ が確認された．最も多くのプロジェクトで共通している ‘E402’ は，選定学習によって適合率が向上した 37 プロジェクト中，半数以下の 15 プロジェクトでのみ出現を確認した．つまり，適合率の向上したプロジェクト群と低下したプロジェクト群で，出現するコーディング規約に明確な特徴は確認できなかった．

次に，適合率が向上した要因を検討するため，説明変数の差異に着目した分析を行った．適合率が改善したプロジェクト群と低下したプロジェクト群を対象に，マン・ホイットニーの U 検定（有意水準 0.05）を用いた有意差検定を実施したところ,「正例数」および「修正率」の 2 指標において統計的な有意差が確認された．検定の結果，正例数における p 値は 0.0124，修正率における

表 5.3: プロジェクトの適合率改善状況別のサンプル数・正例数・負例数の統計比較

指標	適合率が改善したプロジェクト	適合率が低下したプロジェクト
平均総サンプル数	1147.6	1741.2
中央値総サンプル数	853.0	1230.5
平均正例数	302.8	222.2
中央値正例数	293.0	164.5
平均負例数	844.8	1519.0
中央値負例数	635.0	928.5

表 5.4: プロジェクトの適合率改善状況別の修正率の統計比較

指標	適合率が改善したプロジェクト	適合率が低下したプロジェクト
平均修正率	0.344	0.225
中央値修正率	0.350	0.201
標準偏差	0.169	0.157
最小値	0.038	0.022
最大値	0.797	0.573

p 値は 0.0137 であり，いずれも有意水準 0.05 未満であった．表 5.3 および表 5.4 に，適合率が向上した群と低下した群における各統計量を示す．適合率が改善したプロジェクト群は，低下した群と比較して正例数が有意に多く，かつ修正率も有意に高い傾向にあることが示された．この結果は，モデルの学習において十分な正例数を確保することの重要性と，修正率の高さが適合率の向上に寄与していることを示唆している．

一方で，データの構成比にも重要な知見が得られた．正例数そのものの多寡だけでなく，それに対する負例数の多さも適合率に悪影響を及ぼす阻害要因となっている可能性が示唆された．以上の分析結果から，提案手法においては，正例数が十分に存在し，かつ修正率が適切な範囲にあるプロジェクトを学習データとして混合することで，コールドスタート問題の解消と予測性能の向上が実現されることが示唆された．

続いて，学習データの正例数が 100 件から 500 件のプロジェクトのうち，選定学習（欠損値補完あり）で適合率が改善した群と低下した群における，採用しているジャッカード係数の閾値を図 5.1 に示す．図に示す通り，適合率が低下した群は改善した群に比べて，採用しているジャッカード係数の閾値が高い．つまり，適合率が改善したプロジェクトは，低下したプロジェクトと比較して，類似判定のジャッカード閾値を低く設定し，学習データの絶対量の確保が重要であることが示唆される．

5.3.2 全学習で予測性能が改善するプロジェクトの特徴

全学習手法によって単一学習から適合率が改善した，学習データの正例数が 100 件から 500 件の範囲にあるプロジェクトは，向上したプロジェクトが 40 件，低下したプロジェクトが 17 件であった．改善したプロジェクト数は選定学習（欠損値補完あり）の 37 件を上回り，全学習手法の

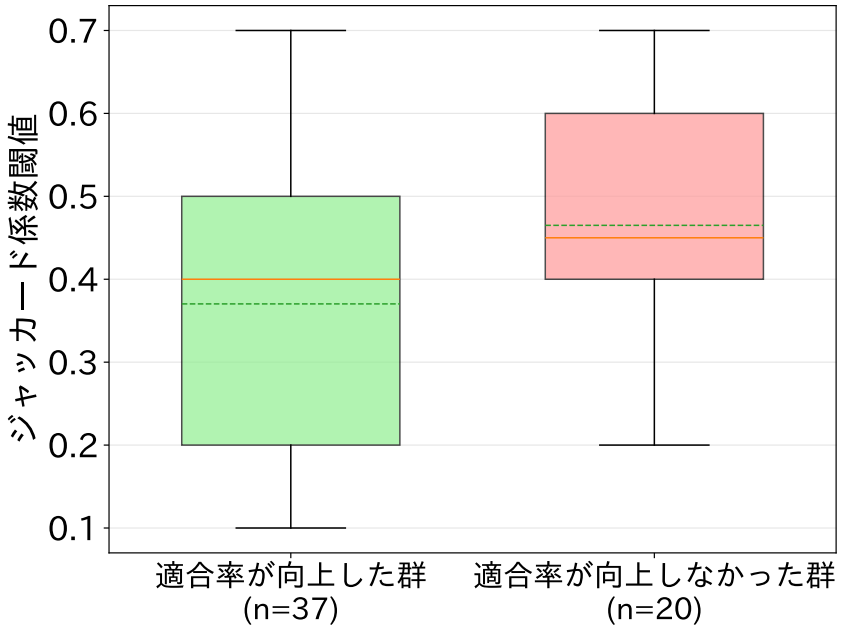

図 5.1: 選定学習のよる適合率向上・非向上プロジェクトのジャッカード係数の箱ひげ図

方がより多くのプロジェクトで適合率の改善が見られた．

選定学習では，学習データの正例数と修正率において，向上群と低下群の間に統計的有意差が確認された．そこで，全学習手法においても同様の分析を実施し，向上群と低下群の特徴を比較した．学習データの正例数については，向上群と低下群の分布に対してマン・ホイットニーの U 検定（有意水準 0.05）を実施したところ，p 値は 0.60 であり，統計的有意差は確認できなかった．修正率についても同様にマン・ホイットニーの U 検定を実施したところ，p 値は 0.09 であり，有意水準 0.05 未満ではなかったため，統計的有意差は確認できなかった．

これらの結果から，全学習手法では選定学習とは異なり，正例数や修正率といったプロジェクト固有の特徴に依存せず，より広範なプロジェクトに対して適合率の改善効果が得られることが示唆された．つまり，選定学習が特定の条件下（正例数や修正率が高いプロジェクト）において精度向上を実現するのに対し，全学習はプロジェクトの特性に関わらず，大規模な学習データの混合による汎用的な効果により適合率を向上させる可能性が示された．

5.4 まとめ

本章では，RQ2 として提案手法である複数プロジェクトのデータを学習に用いることで，予測性能に向上が見られるプロジェクトの特徴を明らかにした．

まず，学習データ内の正例数と修正率に統計的有意差があることが確認された（正例数：p 値 0.0124，修正率：p 値 0.0137）．適合率が向上したプロジェクト群は，低下した群と比較して正例数が有意に多く，かつ修正率も有意に高い傾向にある．この結果は，モデルの学習において十分な正例数を確保することの重要性と，修正率の高さが適合率の向上に寄与していることを示唆している．一方で，発生しているコーディング規約の種類には，向上群と低下群の間で明確な特徴は確認できなかった．また，選定学習で各プロジェクトごとに採用されているジャッカード係数の

閾値を分析した結果，向上群では低下群と比較して閾値が低く設定される傾向にあった．これは，類似判定の閾値を低く設定し，広範囲にデータを学習に取り入れることが，予測性能の向上に有効であることを示唆している．

全学習手法の分析結果では，選定学習で確認された正例数と修正率における向上群と低下群の間の統計的有意差は確認されなかった（正例数：p 値 0.60，修正率：p 値 0.09）．全学習手法では，選定学習（欠損値補完あり）より多くのプロジェクトで適合率の向上が見られた．これらの結果から，全学習手法においては，選定学習とは異なり，正例数や修正率といった特定のプロジェクト特性に依存せず，大規模な学習データの混合による汎用的な効果により，より広範なプロジェクトに対して予測性能の向上を実現できることが示唆された．

以上より，予測性能向上に寄与するプロジェクト特徴は手法によって異なることが明らかになった．選定学習では，正例数が多く修正率が高いという特定のプロジェクト特徴が予測性能の向上に寄与する一方で，全学習では特定のプロジェクト特徴に依存せず，より汎用的に予測性能の向上が期待できることが示された．

第6章　RQ3: 複数プロジェクトを混合する提案手法によりコーディング規約違反の種類ごとに予測性能は異なるか

6.1　概要

本研究が提案する，学習データに予測対象プロジェクト以外の複数プロジェクトの修正履歴を混合することにより，コーディング規約の種類別の予測性能を分析する．予測結果の変化を分析することにより，単一学習と比較して，複数プロジェクトのデータを用いることで正解率の向上するコーディング規約の種類の条件の分析を行う．

6.2　分析方法

分析対象とするプロジェクトは，RQ1 の結果から提案手法による予測性能の変化が大きかった，学習データの正例数が 100 件から 500 件の範囲にあるプロジェクトとする．

分析対象とするコーディング規約違反の種類は，以下の条件を満たすものとする：

- データセット内の複数のプロジェクトで検出されている違反であること
- 学習データの正例数が 100 件から 500 件のプロジェクト全体において，当該規約違反の発生数が 200 件以上であること

本研究では，各規約違反の種類ごとに，単一学習，全学習，選定学習（欠損値補完あり）の 3 手法による予測結果を比較するため，各手法で十分な予測結果が得られる規模として 200 件以上を基準とした．

コーディング規約違反の種類ごとの予測性能の評価には，正解率と F1 値を利用する．正解率を主要な評価指標として採用した理由は，コーディング規約違反の修正要否予測において，修正すべき違反（正例）と修正不要な違反（負例）の両方を正確に判定できることが重要であるためである．一方で，コーディング規約違反の修正データは負例が多い不均衡データであるため，正解率のみでは正例の予測性能を適切に評価できない可能性がある．そのため，正例の予測性能を評価する指標として F1 値も併せて分析する．

各コーディング規約違反の種類ごとの予測性能を算出するため，以下の手順でデータを集計する．

1. 学習データの正例数が 100 件から 500 件の範囲にある全プロジェクトの予測結果を取得する
2. 各プロジェクトの予測結果を，コーディング規約違反の種類（Violation ID）ごとに分類する
3. 同一の Violation ID を持つ予測結果を，プロジェクト横断的に統合する

4. 統合したデータに対して，各評価指標（正解率，F1 値）を算出する

各規約違反の種類において，3 手法のうち最も高い正解率を達成した手法を特定し，その手法を当該規約違反の種類において最も有効な手法とする．また，各手法が最も高い正解率を達成した規約違反の種類数を集計し，手法間の比較を行う．正解率が最大値を取った手法ごとに，コーディング規約違反の種類を 3 つのグループに分割する．

- グループ 1：単一学習で正解率が最も高かった規約違反の種類
- グループ 2：全学習で正解率が最も高かった規約違反の種類
- グループ 3：選定学習（欠損値補完あり）で正解率が最も高かった規約違反の種類

各グループごとに，以下の指標を分析する

- 当該グループに属するコーディング規約違反の種類数
- 各規約違反の種類が発生しているプロジェクト数（発生プロジェクト数）
- 発生プロジェクト数の中央値

発生プロジェクト数は，学習データの正例数が 100 件から 500 件のプロジェクト中で，当該コーディング規約違反が発生しているプロジェクトの数を示す．この指標により，提案手法が有効に働くコーディング規約違反の種類が，複数プロジェクトで共通して発生している規約であるかを分析する．

6.3 結果

6.3.1 3 手法間のコーディング規約ごとの正解率の評価

表 6.1 に学習データの正例数が 100 件から 500 件のプロジェクトの，手法別のコーディング規約の種類ごとの正解率と F1 値を示す．表のコーディング規約の種類は，比較する手法内で最も高い正解率を取った手法ごとに分割している．各手法内の順序は，改善幅の降順に並んでいる．また，表中のそれぞれの規約の種類において，最も予測性能が高かった手法の正解率を太字で示す．

正解率が最大値を取ったコーディング規約の種類数は，重複を許して単一学習では 3 種類，全学習では 9 種類，選定学習（欠損値補完あり）では 7 種類であった．コーディング規約別の予測結果では，‘E111’ と ‘F821’ を除くすべての規約において，全学習または選定学習が単一学習より高い正解率を示した．一方，F1 値の変化は正解率の変化と比較して小さい．特に単一学習と選定学習の間では，多くの規約で F1 値の差が 5 ポイント以下である．

例えば ‘E128’ の結果では，単一学習から全学習によって正解率が 10 ポイント増加しているが，F1 値の変化は 1 ポイント未満である．このような結果はコーディング規約の修正データが，負例が多い不均衡データであることに起因する．‘E128’ の予測結果は，提案手法によって予測可能な正例数に大きな変化は見られなかったが，True Negative の件数が増加していた．つまり，複数プロジェクトのデータを学習に使用する提案手法は，単一学習と比較して，修正されないコーディング規約違反を修正不要であると適切に予測する傾向にある．

表 6.1: コーディング規約違反の種類ごとの学習手法の比較

規約違反 ID	正解率			F1 値			発生プロジェクト数	発生総数
	単一学習	全学習	選定学習	単一学習	全学習	選定学習		
F821	**0.76**	0.66	0.69	**0.37**	0.28	0.31	9	375
E261	**0.82**	0.78	0.81	**0.34**	0.13	0.26	5	273
E111	**0.95**	0.8864	**0.95**	**0.23**	0.17	**0.23**	3	957
E502	0.65	**0.89**	0.65	**0.15**	0.13	0.15	4	219
E303	0.54	**0.74**	0.59	0.24	**0.30**	0.23	16	333
W291	0.59	**0.75**	0.59	0.04	**0.08**	0.04	7	255
E128	0.72	**0.82**	0.73	0.13	**0.13**	**0.13**	12	2,193
E721	0.66	**0.75**	0.66	0.34	0.33	**0.35**	9	279
E127	0.74	**0.81**	0.73	**0.31**	0.30	0.29	9	1,803
E203	0.74	**0.79**	0.74	**0.17**	0.16	0.16	23	1,125
E501	0.74	**0.77**	0.76	0.48	0.49	**0.49**	57	40,098
E231	0.62	**0.65**	0.62	0.21	0.15	**0.22**	11	2,142
F405	0.63	0.62	**0.75**	0.47	0.47	**0.55**	4	1,023
E225	0.59	0.59	**0.71**	0.35	0.22	**0.35**	9	678
E402	0.86	0.89	**0.89**	0.13	0.13	**0.14**	13	738
F841	0.77	0.64	**0.80**	**0.34**	0.18	0.33	14	426
E302	0.58	0.57	**0.60**	**0.13**	0.09	0.13	17	480
F401	0.86	0.83	**0.87**	0.13	0.04	**0.15**	29	1,101

予測対象プロジェクト内のすべてのコーディング規約違反の修正要否予測結果を見ると，発生しているコーディング規約違反の大多数を占める ‘E501’ の予測結果に大きな影響を受ける．学習データ内の正例数が 100 件から 500 件以内のプロジェクトにおいても，‘E501’ は最も多い 40,098 件発生しており，次に多い ‘E128’ は 2,193 件しか発生しておらず，約 18 倍の差がある．そのため，全体の予測結果の変化幅が小さく見られた．しかし，コーディング規約別の結果を見ると，提案手法である学習データに予測対象以外の修正履歴を混合する手法は，発生頻度の低いコーディング規約の修正要否予測において正解率を改善させる．

全学習と選定学習間の予測結果を比較すると，全学習手法では単一学習から ‘E303’ の正解率が 19 ポイント上昇している一方で，‘F841’ では 13 ポイント低下している．選定学習では，単一学習から最も正解率が改善した規約は ‘F405’ で，12 ポイント改善している．改善幅では全学習の方が選定学習より大きいが，選定学習は単一学習より正解率が劣っているコーディング規約が ‘F821’ のみであり，その他規約については同じ正解率か選定学習の方が優れている．

6.3.2 正解率が変化するグループごとの分析

表 6.1 中のグループは上から単一学習，全学習，選定学習で正解率が最も高い値をとったコーディング規約のグループである．発生プロジェクト数は，学習データの正例数が 100 件から 500 件のプロジェクト中で，当該コーディング規約への違反が発生しているプロジェクトの数を示す．各グループごとの発生プロジェクト数の中央値は，単一学習では 5.0，全学習では 12.0，選定学習では 13.5 である．統計的な有意差を示すものではないが，傾向として提案手法によって予測性能

が改善したと判断された全学習と選定学習のグループは，単一学習のグループと比較して発生プロジェクト数が多い．つまり，提案手法によって正解率が改善するコーディング規約の種類は，混合するプロジェクトのデータ内にも同種類のコーディング規約に対する修正履歴が多く含まれているコーディング規約である．混合するプロジェクト数が多いほど，コーディング規約違反の修正要否予測性能が向上する結果は，RQ2 で適合率の改善条件の分析を行った結果とも一致する．

6.3.3 複数プロジェクトのデータを用いることで予測性能が向上するコーディング規約

前節では，コーディング規約が共通して発生しているプロジェクト数が多いほど，予測性能が向上する傾向が見られた．そこで本節では，共通プロジェクトが十分に存在する場合に，提案手法である全学習手法と選定学習手法のどちらが予測性能を向上させるかを分析する．全学習によって単一学習から正解率が 10 ポイント以上向上した規約は，‘E502’，‘E303’，‘W291’，‘128’ であり，選定学習では ‘F405’ と ‘E225’ であった．

全学習と選定学習の結果の違いを分析するために，それぞれの規約に対して，類似プロジェクトの分析を行った．分析方法としては，選定学習により，予測対象プロジェクトの修正要否予測を行う際に混合したプロジェクト数をカウントした．結果として，選定学習で予測性能が高かったコーディング規約では，類似プロジェクトの平均数が 14.8 プロジェクトであった．全学習手法で予測性能が高かったコーディング規約では，類似プロジェクトの平均数が 6.9 プロジェクトであった．選定学習では全学習と比較して，類似プロジェクト数が 2 倍程度多いことが示された．つまり，予測対象コーディング規約が他プロジェクトでも発生している場合において，選定学習と全学習の予測性能の差は，類似プロジェクト数の差によって説明される．選定学習による，修正傾向の類似しているプロジェクトのみで学習データの拡張が可能な場合，すべてのデータを学習する全学習手法より優れた学習が可能であることが示唆される．一方で，予測対象コーディング規約が共通しているプロジェクト数が十分にある場合でも，修正傾向の類似しているプロジェクトが少ない場合，混合先のプロジェクトの絞り込みを行わずにすべてのプロジェクトの修正履歴を混合することにより，単一学習からの正解率の向上が期待される．

6.4 まとめ

本節では，コーディング規約違反の種類ごとに，提案手法による予測性能の変化を分析した．学習データの正例数が 100 件から 500 件のプロジェクトにおける，18 種類のコーディング規約違反に対する予測結果を分析した結果，以下の知見が得られた．

まず，コーディング規約違反の種類ごとの予測性能において，提案手法である全学習および選定学習は，単一学習と比較して多くの規約で高い正解率を達成した．具体的には，全学習が 9 種類の規約で正解率の最大値を取り，選定学習が 7 種類の規約で最大値を取ったのに対し，単一学習が最大値を取った規約は 3 種類のみであった．特に，‘E111’ と ‘F821’ を除くすべてのコーディング規約において，全学習または選定学習が単一学習より高い正解率を示し，提案手法は発生頻度の低いコーディング規約の修正要否予測において，正解率を改善させることが明らかになった．

次に，正解率が最大値を取った手法ごとにコーディング規約を 3 つのグループに分割して分析した結果，提案手法によって予測性能が改善したグループは，単一学習が最も高い正解率を示し

たグループと比較して，発生プロジェクト数の中央値が大きいことが判明した．各グループの発生プロジェクト数の中央値は，単一学習では 5.0，全学習では 12.0，選定学習では 13.5 であった．この結果は，提案手法によって正解率が改善するコーディング規約の種類は，混合するプロジェクトのデータ内にも同種類のコーディング規約に対する修正履歴が多く含まれているコーディング規約であることを示している．この知見は，RQ2 で分析した適合率の改善条件と一致しており，複数プロジェクトのデータを学習に使用する提案手法は，混合するプロジェクト内に同種類の修正履歴が多く存在するコーディング規約において，特に効果的であることが示された．

最後に，全学習と選定学習の予測性能の違いを分析した結果，選定学習で予測性能が高かったコーディング規約では，類似プロジェクトの平均数が 14.8 プロジェクトであったのに対し，全学習で予測性能が高かったコーディング規約では，類似プロジェクトの平均数が 6.9 プロジェクトであった．選定学習では全学習と比較して，類似プロジェクト数が約 2.1 倍多いことが示された．この結果は，予測対象コーディング規約が他プロジェクトでも発生している場合において，選定学習と全学習の予測性能の差は，類似プロジェクト数の差によって説明されることを示している．

ここまでの分析結果から，コーディング規約違反の修正要否予測において，単一学習，全学習，選定学習の 3 つの手法間における予測性能の差異は，当該規約違反が共通して発生しているプロジェクト数と，その中で修正傾向が類似しているプロジェクト数によって決定されることが明らかになった．

第7章　考察

7.1　概要

本章では，提案手法による予測性能の改善について，複数の観点から考察を行う．まず，類似プロジェクトの集約による学習データの拡張が適合率向上に与えた影響について分析する．次に，類似度測定方法の妥当性を検証し，提案手法が適切に類似プロジェクトを収集できているかを確認する．さらに，コーディング規約の種類による提案手法の効果の違いについて考察し，最後に，大規模言語モデルを用いた予測手法との比較を通じて，提案手法の有効性を評価する．

7.2　類似プロジェクトの集約による学習データの拡張と適合率向上

本節では，類似プロジェクトの集約による学習データの拡張が適合率向上に与えた影響について分析する．学習データを混合した結果，正例数が少ない（100 件から 500 件程度）プロジェクトにおいて，57 プロジェクト中 37 プロジェクトにおいて選定学習手法により適合率が改善することが確認された．この精度の改善をもたらした主要な要因は，類似プロジェクトの集約による「学習データ量の確保」と，それに伴う「修正傾向の類似した負例データの拡張」にあると考えられる．

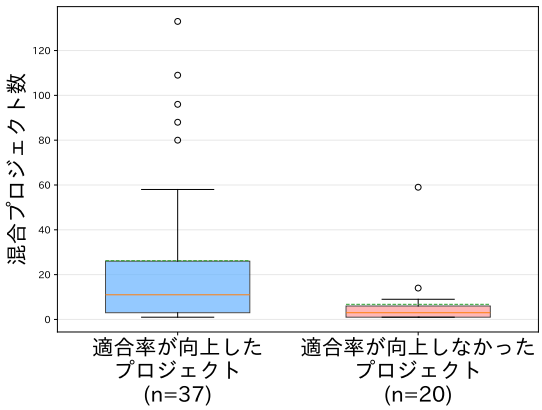

図 7.1: 適合率が向上した群と低下した群における混合プロジェクト数

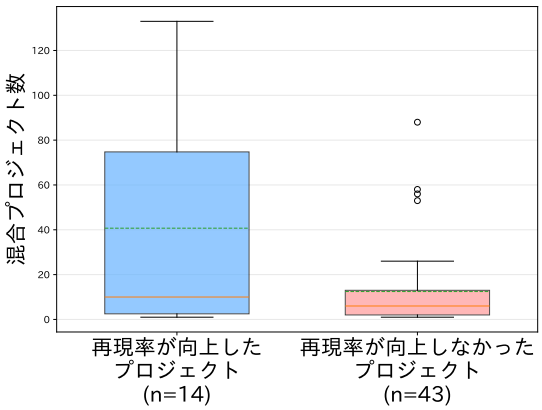

図 7.2: 再現率が向上した群と低下した群における混合プロジェクト数

図 7.1 は，学習データの正例数が 100 件から 500 件の範囲にあるプロジェクトを対象とし，適合率が向上した群と低下した群に分けた際の，混合プロジェクト数に関する箱ひげ図である．同様に，図 7.2 には再現率に関する結果を示す．

適合率の変動について分析を行ったところ，向上群と低下群の間で，混合したプロジェクト数にマン・ホイットニーの U 検定（有意水準 0.1）による有意差が見られた．この結果は，単一プロジェクトの履歴のみでは不足していた学習データを，他プロジェクトから補完することが有効であ

ることを裏付けている．再現率についても p 値は 0.0994 であり，有意差が確認された．また，クリフのデルタは 0.30 と小さい効果量を示した．これは，予測性能が低下したプロジェクトにおいては，学習に寄与する類似プロジェクトを十分に収集できなかったことが精度の改善を阻んだ要因であることを示唆している．

一般に，コーディング規約違反のデータセットは，修正されない違反（負例）が多数を占め，実際に修正される違反（正例）が 3 割程度に留まる不均衡の問題を抱えている．単一プロジェクトの学習データでは，正例が極端に少ない状態ではモデルの学習が正常に行われず，予測性能が低下する．本手法によって，予測対象プロジェクトとコーディング規約違反に対する修正履歴の類似性の高いプロジェクトから，正例データと修正傾向が類似した負例データの両方を混合できたことで，学習データ量が確保されるとともに，モデルが「修正すべきでない違反」の特徴をより詳細に学習できるようになった．特に，修正傾向が類似した負例データを混合することで，予測時に False Positive（修正すべきでない違反を修正すべきと誤判定すること）が減少し，結果として適合率の有意な向上に寄与したものと推察される．

7.3 類似度測定方法の妥当性の評価

本節では，提案方法によって，類似プロジェクトが適切に収集できているかを検証する．本研究では，予測対象プロジェクトと類似しているプロジェクトを選定するために，ジャッカード係数とユークリッド距離の 2 種類の値を用いて，類似プロジェクトを決定している．2 種類の値を利用することによって，発生しているコーディング規約違反の種類の類似度と，違反に対する修正率の類似度を測ることが可能である．類似度測定の妥当性を検証するため，選定学習（欠損値補完なし）と選定学習（欠損値補完あり）で，類似していると判断するためのジャッカード係数とユークリッド距離の閾値を最も厳しくした場合のプロジェクトの混合関係を見ることで，妥当なプロジェクトが収集されているかを確認する．最も厳しい類似度判定を行うための閾値を採用する理由は，最も厳しい条件で選定されたプロジェクトの関係性を確認することで，類似度測定が妥当なプロジェクトを収集できているかを検証できるためである．本研究において最も厳しい類似判定を行う，ジャッカード係数とユークリッド距離の閾値は，ジャッカード係数が 1，ユークリッド距離が 0 の場合である．

最も閾値を厳しく設定した場合の選定学習で，予測モデルの構築に複数プロジェクトの学習データを利用していたプロジェクトの混合関係を表 7.1 に示す．表中の最も左の列には予測対象プロジェクトを示し，そのプロジェクトと類似していると判断されたプロジェクトのうち，修正率の欠損値補完を行わない手法を二列目，修正率の欠損値補完を行う手法を三列目に示す．

表の結果から欠損値補完を行わない手法のほうが混合プロジェクト数が多いことがわかる．これは，欠損値補完を行わない選定学習手法のユークリッド距離の計算時に，どちらかのプロジェクトでしか発生していないコーディング規約違反があった場合，その修正率はユークリッド距離の計算から除外しているため，ユークリッド距離が小さくなり，類似していると判定されることが原因である．

表中のプロジェクト間の混合関係について詳しく評価する．表の最初の混合関係である，click[1]

[1] `https://github.com/pallets/click`

表 7.1: 予測対象プロジェクトと類似プロジェクトの比較

予測対象プロジェクト	選定学習（欠損値補完なし）	選定学習（欠損値補完あり）
Jinja2	click	click
click	Jinja2	Jinja2
pytest-cov	tmuxp	(なし)
nbconvert	aiortc	(なし)
pelican	apispec, awacs	apispec
pytest-benchmark	aiosmtplib, mcstatus, openai-whisper, xhtml2pdf	(なし)
apispec	awacs, pelican	pelican
aiortc	nbconvert	(なし)
aiosmtplib	mcstatus, openai-whisper, pytest-benchmark, xhtml2pdf	mcstatus, openai-whisper, xhtml2pdf
openai-whisper	aiosmtplib, mcstatus, pytest-benchmark, xhtml2pdf	aiosmtplib, xhtml2pdf
xhtml2pdf	aiosmtplib, mcstatus, openai-whisper, pytest-benchmark	aiosmtplib, mcstatus, openai-whisper
tmuxp	pytest-cov	(なし)
mcstatus	aiosmtplib, openai-whisper, pytest-benchmark, xhtml2pdf	aiosmtplib, xhtml2pdf
awacs	apispec, pelican	(なし)

と Jinja2[2]は互いに混合関係にある．これらは共に，Pallets プロジェクトという同一の開発元によって開発，保守されており，コーディングスタイルの共通点が高いプロジェクトとして，選定学習の目的通りに類似プロジェクトが収集できていた．これら 2 プロジェクトは互いの修正履歴を混合することで，予測性能が僅かに向上した．

欠損値補完を行わない選定学習手法では，xhtml2pdf，aiosmtplib，mcstatus，openai-whisper，pytest-benchmark の 5 プロジェクトが複数回にわたって互いに混合関係にある．この 5 プロジェクトのコーディング規約違反の修正履歴を確認したところ，データには ‘E501’ という，他プロジェクトでも最も多く発生している，行内の文字数制限に関するコーディング規約違反の他には，数種類のコーディング規約違反のみ存在する．欠損値補完を行わないプロジェクトの類似度測定では，発生しているコーディング規約の種類数が少ない場合に，ユークリッド距離が必然的に高くなっていた．つまり，欠損値補完を行わない選定学習手法において，本来類似していないプロジェクト同士が混合される要因は，計算対象となる次元数が異なる問題に起因する．具体的には，発生している規約の種類が極めて少ないプロジェクト間では，距離計算に使用される次元数が著しく低下する．そのため，修正傾向の異なるプロジェクトであっても，共通して発生している規約のみにユークリッド距離の計算が行われるため，修正傾向が類似していると判定される．

一方で，欠損値補完を行う類似度測定では，混合プロジェクトが少なくなってしまっているが，先述の発生しているコーディング規約違反の種類が少ないことに起因する，予期しないプロジェクトの混合を抑制できていることがわかる．つまり，欠損値補完を用いない類似プロジェクト測定結果と欠損値補完を用いる類似プロジェクト測定結果を比較すると，本研究の目的である，各コーディング規約違反の種類に対する修正率が類似するプロジェクトの収集において，欠損値補完を用いる手法の方が目的に適していることが確認できる．この結果は，類似プロジェクトの測定方法の違いに起因するものであり，採用する閾値の設定に依存しない，本研究で提案した手法の特徴であることが示唆される．

7.4 コーディング規約の種類による提案手法による予測結果への影響

表 6.1 の結果から，複数プロジェクトの修正履歴を混合する提案手法は，複数プロジェクトで規約違反が共通して発生しているプロジェクトで予測結果が向上する傾向が見られた．しかし，‘E502’ や ‘F405’ では発生プロジェクト数がグループ 1 の規約と同程度であるが，正解率が提案手法によって 10 ポイント以上上昇している．‘E502’ と ‘F405’ の規約違反内容を以下に示す．

- E502：コードの括弧内に不必要なバックスラッシュが存在する
- F405：from module import * を使用している

‘E502’ に定義される，括弧内のバックスラッシュは Python 言語において構造的に不必要である．他言語においては，括弧内のバックスラッシュは構造的に必要な場合があるが，Python 言語においては構造的に不必要である．‘F405’ に定義される，Python 言語において，モジュールから全ての関数をインポートするスターインポートは，モダンな開発では推奨されていない．これは，利用しない関数やクラスがインポートされることで，名前空間の衝突や定義不一致の原因となる

[2] https://github.com/pallets/jinja

ためである．これらの規約違反は，開発者によるコーディングスタイルの違いによらず，コードの可読性を低下させる要因である．つまり，‘E502’ や ‘F405’ は，共通して発生しているプロジェクト数が単一学習の方が高い正解率であった規約と同程度であるが，修正傾向がプロジェクトによらず共通しており，提案手法によって正解率が向上したと考えられる．

次に，他規約と同程度のプロジェクトに存在する規約であるが，提案手法によって正解率が向上しなかった規約について考察する．表 6.1 の結果から，正解率が提案手法によって改善しなかった規約は ‘E111’，‘E261’，‘F821’ である．‘E111’．‘E261’．‘F821’ の規約違反内容を以下に示す．

- E111：インデントのスペースの数が 4 の倍数でない
- E261：インラインコメントの前には最低 2 つのスペースが必要である
- F821：定義されていない変数名の参照をしている

‘E111’ に定義される，インデントのためのスペース数は，Python 言語では 4 の倍数であることを推奨している．しかし，コードベースが古い場合や，標準ではない特定のコードスタイルに準拠している場合，プロジェクトによってはインデントにスペースの数を 2 の倍数に設定することがある．‘E261’ に定義される，インラインコメントの前にスペースが 2 つ必要である規約は，放置していてもエラーにならず，コメントであるため，コードの可読性にも直接影響しない重要度の低いコーディング規約である．そのため，‘E261’ の修正は周辺コードの修正に併せて修正されることが多く，プロジェクト間で修正傾向が異なると考えられる．‘F821’ に定義される，定義されていない変数名の参照は，バグのように見えるが，メタプログラミングの副作用により，静的解析ツールがコーディング規約への違反であると検出してしまう．例えば，フレームワークの利用によって，当該ファイルに定義されていない変数であっても安全に使用することが可能であり，このような事例は修正する必要がない違反である．つまり，‘F821’ は常に修正する必要がある規約ではないため，プロジェクトによって修正傾向が異なると考えられる．以上の結果から，提案手法によって正解率が向上しなかった規約は，プロジェクトによって修正傾向が異なる規約であり，提案手法によって正解率が向上しなかったと考えられる．

7.5 提案手法は大規模言語モデルより優れた修正要否予測が可能か

Python の静的解析ツールの検出結果の修正要否予測において，大規模言語モデルが利用できるかを調査した．大規模言語モデルを用いた修正要否予測を行うにあたって，従来研究を基に近似実験を行う [24]．従来研究では，対象言語に Java を選択しており，対象とする静的解析によるエラーも 2 種類のみに絞っている．そのため，本研究で検証を行う際には，対象言語を Python に変更し，対象アラートも Flake8 をカバーできるようにプロンプトを変更した．予測に使用するプロンプトを Listing7.1 に示す．プロンプトには，従来研究で予測性能が最も高かった Few-shot を利用する．利用する大規模言語モデルには，Openai 社の GPT-5 mini を利用する．GPT-5 mini は GPT-5 と比較して，生成精度は低下する．しかし，従来研究で使用されている GPT-4 と GPT-3.5 Turbo より高い精度で生成が可能であるため，より低コストで利用可能な GPT-5 mini を利用する．また，大規模言語モデルには，修正要否の予測結果と同時に，その判定結果となった理由も出力させる．

表 7.2: 大規模言語モデルによる規約違反修正要否予測性能の比較

プロジェクト名	規約違反数	正例数	負例数	予想正例数	予想負例数	正解率
awscli	693	107	586	683	10	0.16
meson	213	105	108	210	3	0.50
py-cord	223	101	122	220	3	0.46
Telethon	420	74	346	354	66	0.26
gseapy	312	122	179	292	20	0.41

検証に使用するプロジェクトはデータセットから，学習データの正例数が 100 件から 500 件のプロジェクトで，修正履歴に存在するコーディング規約違反の種類が多いものから 5 プロジェクトを採用する．具体的なプロジェクト名は ‘awscli’，‘meson’，‘py-cord’，‘Telethon’，‘gseapy’ の 5 プロジェクトである．検証には，プロジェクトのデータを学習用とテスト用に 4:1 の割合で分割したテストデータのみを用いる．

表 7.2 に大規模言語モデルによる，コーディング規約違反修正要否予測の結果の概要を示す．表の結果から，予想正例数が予想負例数に比べて非常に多いことがわかる．正解率の高いプロジェクトは，正例数の占める割合が高いプロジェクトであり，プロジェクトごとに予測性能が変化する結果は見られない．本来，規約違反の修正要否は，放置される方が多いため，大規模言語モデルは開発現場におけるコーディング規約違反への対象方法を捉えられていない．

大規模言語モデルによる予測結果を目視により確認したところ，大規模言語モデルが負例（修正する必要がない）と判定する規約違反には共通性が見られた．大規模言語モデルが，負例と予測できた規約違反は，全て ‘E501’ という種類の規約に関する修正要否予測のみであった．大規模言語モデルによる修正要否予測結果の理由を目視したところ，‘E501’ の中でも文字数が超過しているコンテキストが，docstring やコメントである場合に大規模言語モデルは，その違反を負例と予測していた．docstring とは Python 言語でクラスや関数，モジュールなどのコードに記述する，コードの目的や処理内容を説明するための文章である．これは，few-shot プロンプト内の例示において，修正が不要（FALSE）の例としてコメントの E501 のみが提示されていたため，大規模言語モデルはコメントの文字数超過以外のコーディング規約違反を全て修正が必要と判定する傾向を示したと考えられる．つまり大規模言語モデルは，few-shot で指定しない大部分の規約違反については，すべて修正が必要と判定する傾向を示したと考えられる．この結果はプログラミング言語による違いが原因ではなく，従来研究が対象としてきた静的解析ツールの検出結果が誤検出かどうかという明確な目的変数とは異なり，本研究では修正要否というプロジェクトや開発者によって判断が異なる曖昧な目的変数を予測しようとしている点に起因すると考えられる．修正要否は，誤検出の判定とは異なり，プロジェクトの開発方針や開発者の価値観に依存する主観的な判断であるため，単純な few-shot プロンプトによる例示だけでは，その多様性を捉えることが困難である．

この判定結果では開発者のレビューコストの削減にはつながらず，本研究の趣旨と一致していない結果である．そのため，静的解析ツールの検出結果の修正要否分類には大規模言語モデルの利用より，従来通りの機械学習モデルを利用した予測が適切であることが示唆される．

Listing 7.1: Prediction Prompt

```
You are an experienced Python developer. Analyze the warnings reported by the static
    analysis tool (flake8) and determine whether the code actually needs to be fixed.

Let's look at some examples:

---

Example 1:
Q: Look at this code. flake8 is reporting "E501: line too long (120 > 79 characters)".

```python
def process_data(data):
    if data is not None:
        return data.value
    return None
```

A: This warning should be fixed. The line exceeds the maximum length limit recommended
     by PEP 8. To improve code readability, the long line should be split.
Answer: TRUE

Example 2:
Q: flake8 is reporting "F401: 'os' imported but unused". What do you think about this
    code?

```python
import os
import sys

def main():
    print(sys.argv[0])
```

A: Looking at the code, the `os` module is imported but never actually used. Unused
    imports clutter the codebase and should be removed.
Answer: TRUE

Example 3:
Q: There's a warning "E501: line too long (95 > 79 characters)". How would you judge
    this?

```python
# This is a very long comment that explains the complex algorithm used in this
    function
def complex_function():
    pass
```
```

```
```

A: In this case, the long line is a comment. Long comment lines are often acceptable
    in real-world development, and flake8 can be configured to ignore them. Since
    there's no functional issue, this doesn't need to be fixed.
Answer: FALSE

Now, analyze the actual code:

File: {warning.file_path}
Line number: {warning.line_number}
Violation ID: {warning.violation_id}
Violation message: {warning.violation_message}

Problematic code:
```python
{warning.code_snippet}
```

Surrounding context:
```python
{warning.context}
```

Q: Regarding this code, flake8 has reported a warning "{warning.violation_id}: {
    warning.violation_message}". Should this warning actually be fixed, or is it
    acceptable to ignore?

A: [Please respond in the following format]
- Write "TRUE" if the warning should be fixed, and explain why.
- Write "FALSE" if the warning can be ignored, and explain why.
```
```

第8章　妥当性への脅威

8.1　内的妥当性

目的変数の計測において，コーディング規約に違反しているコードが修正された場合と，削除された場合を修正としてとらえ，正例として計測している．コーディング規約違反の中には，単純に削除するのみで解消されるエラーが存在するため，削除も正例と計測している．しかし，コーディング規約に違反しているコードが単に不要になり削除された場合も正例と計測している．コードの移動に関しても GitHub の仕様上，削除と追加という扱いになるため，コードの移動によってコーディング規約に違反しているコードが正例として計測されている可能性がある．

本研究で用いた機械学習アルゴリズム以外にも，より正確な予測を可能にするアルゴリズムが存在する可能性がある．しかし，本研究において検証した，予測対象プロジェクト以外のデータを学習に使用する手法は，少量の学習データを他プロジェクトのデータで補完する手法であるため，他の機械学習アルゴリズムによる予測を行う際でも効果的に働くと考えられる．それぞれの機械学習モデルのパラメータの更新回数である，イテレーション回数を 10,000 に設定したが，データサイズが大きいプロジェクトではモデルが収束しないことが確認された．モデルが収束しない場面も確認されたが，本研究で主に用いた実験環境とは別の環境で，複数回実行した場合でも予測結果に変化はなかったため，モデルが収束しない問題に関しては，本研究の結果に対して大きな影響を与える可能性は低い．

LLM の予測性能はプロンプトの設計（Prompt Engineering）に大きく依存するため，本研究で使用した Few-shot プロンプトが LLM の性能を最大限に引き出しているとは限らない．

8.2　外的妥当性

本研究では検証対象として Python 言語を主な開発言語とした，170 件の GitHub リポジトリを収集して検証を行った．そのため，対象言語を Python 以外の言語とした場合や，検証対象の静的解析ツールを変更した場合，予測結果が変化することが示唆される．プロジェクト数をさらに拡張した場合や，対象とするプロジェクトや期間を変更した場合に予測性能が変化することが示唆される．しかし，本研究で利用したデータセットは，データ数の平均値が 4,264 であり，十分なデータ数を確保できているため，データ数を拡張することによる予測性能の低下の可能性は低いと考えられる．また，選定学習においては，より類似度の高いプロジェクトを混合できるため，更なる予測性能の向上が見込める．

第9章　おわりに

本研究では，静的解析ツールによって検出される膨大な規約違反コードの中から，修正が必要なものを自動分類する手法の改善に取り組んだ．特に，単一プロジェクトでの学習データ不足に起因するコールドスタート問題を解消するため，他プロジェクトの修正履歴を動的に混合して学習する手法を提案し，その有効性を多角的に検証した．

実験を通じて得られた一つ目の知見は，プロジェクト横断的な学習データの活用が，従来手法におけるコールドスタート問題を解消可能である点である．全学習及び選定学習のいずれの手法においても，単一プロジェクトのみを対象とした場合と比較し，適合率の改善が見られた．この傾向は学習データ内の正例数が 100 件から 500 件程度のデータを持つプロジェクトにおいて顕著であり，本手法がデータ不足を補完し，予測モデルの安定化に寄与することを示している．

二つ目に，提案手法が有効に働くプロジェクトの特徴を明らかにした．選定学習において適合率が向上したプロジェクト群は，低下した群と比較して正例数が有意に多く，かつ修正率も有意に高い傾向にあることが確認された．これは，モデルの学習において十分な正例数を確保することの重要性と，修正率の高さが適合率の向上に寄与していることを示唆している．一方で，全学習手法では選定学習とは異なり，正例数や修正率といった特定のプロジェクト特性に依存せず，より広範なプロジェクトに対して適合率の向上効果が得られることが示された．

三つ目に，コーディング規約違反の種類ごとの予測性能の変化を分析した結果，提案手法は多くの規約で正解率を向上させることを明らかにした．提案手法によって正解率が向上するコーディング規約の種類は，混合するプロジェクトのデータ内にも同種類のコーディング規約に対する修正履歴が多く含まれているコーディング規約である．また，選定学習で予測性能が高かったコーディング規約では，類似プロジェクトの平均数が 14.8 プロジェクトであったのに対し，全学習で予測性能が高かったコーディング規約では，類似プロジェクトの平均数が 6.9 プロジェクトであった．この結果は，予測対象コーディング規約が他プロジェクトでも発生している場合において，選定学習と全学習の予測性能の差は，類似プロジェクト数の差によって説明されることを示している．

さらに，大規模言語モデルとの比較実験を通じて，本手法の独自性と実用的な優位性を確認した．汎用的な知識に基づき「過剰修正」を促す傾向にある大規模言語モデルに対し，本手法は過去の修正履歴を学習データとした予測モデルである．これにより，コーディング規約違反を修正しないという現状の判断基準を反映した，現実的な修正要否予測が可能であることを示した．

以上の結果から，本研究の貢献は，複数プロジェクトの履歴混合が，修正履歴の不均衡さを緩和し，判定精度を向上させることを定量的，定性的に実証した点にある．特に，開発現場の履歴に基づいた「現実的かつ文脈依存的な修正予測」の有効性を示したことは，静的解析ツールの実運用におけるノイズ削減と開発効率の向上に向けた重要な知見である．本研究が提示した手法は，リソースの限られた開発初期段階のプロジェクトにおいても高精度な品質管理を実現する補助となり，同分野の研究の進展に寄与するものである．

謝 辞

本研究を進めるにあたり多くの方々に，御指導，御協力，御支援を承りました．ここに御世話になった方々への感謝の意を記します．はじめに，和歌山大学システム工学部伊原彰紀准教授に厚く御礼申し上げます．本研究の遂行において，伊原准教授からは研究の方向性から実験設計，論文執筆に至るまで，終始一貫して丁寧なご指導をいただきました．伊原准教授の御尽力に敬意を表し，心より感謝申し上げます．次に，本研究の主査を務めていただきました大平先生に厚く御礼申し上げます．大平先生には，大ゼミなど普段からご意見をいただき，視野を広げることができました．本研究の審査においても，貴重なご指導とご助言をいただきました．心より感謝申し上げます．また，本研究の副査を務めていただきました吉川先生に厚く御礼申し上げます．本研究の審査において，貴重なご指導とご助言をいただきました．心より感謝申し上げます．さらに，本研究の完成に向けて，貴重な時間を割いて論文執筆にご協力いただきましたソーシャルソフトウェア工学研究室のメンバーの皆様に心より感謝申し上げます．本研究は，皆様のご支援とご協力により完成することができました．心より感謝いたします．

参 考 文 献

[1] Alexander Trautsch. Effects of automated static analysis tools: A multidimensional view on quality evolution. In *Proceedings of the 41st International Conference on Software Engineering (ICSE'19)*, p. 184–185, 2019.

[2] Moritz Beller, Georgios Gousios, and Andy Zaidman. How (much) do developers test? In *Proceedings of the 37th International Conference on Software Engineering (ICSE'15)*, pp. 559–562, 2015.

[3] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. Why don't software developers use static analysis tools to find bugs? In *Proceedings of the 35th International Conference on Software Engineering (ICSE'13)*, pp. 672–681, 2013.

[4] Moritz Beller, Radjino Bholanath, Shane McIntosh, and Andy Zaidman. Analyzing the state of static analysis: A large-scale evaluation in open source software. In *Proceedings of the 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER'16)*, 2016.

[5] Diego Marcilio, Rodrigo Bonifácio, Eduardo Monteiro, Edna Canedo, Welder Luz, and Gustavo Pinto. Are static analysis violations really fixed? a closer look at realistic usage of sonarqube. In *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*, pp. 209–219. IEEE, 2019.

[6] Xueqi Yang, Zhe Yu, Junjie Wang, and Tim Menzies. Understanding static code warnings: An incremental ai approach. *Expert Systems with Applications*, Vol. 167, No. C, April 2021.

[7] Thu Trang Nguyen, Pattaravut Maleehuan, Toshiaki Aoki, Takashi Tomita, and Iori Yamada. Reducing false positives of static analysis for sei cert c coding standard. In *Proceedings of the Joint 7th International Workshop on Conducting Empirical Studies in Industry and 6th International Workshop on Software Engineering Research and Industrial Practice*, p. 41–48, 2019.

[8] Joseph R. Ruthruff, John Penix, J. David Morgenthaler, Sebastian Elbaum, and Gregg Rothermel. Predicting accurate and actionable static analysis warnings: An experimental approach. In *Proceedings of the 30th International Conference on Software Engineering (ICSE'08)*, p. 341–350, 2008.

[9] Lori Flynn, William Snavely, David Svoboda, Nathan VanHoudnos, Richard Qin, Jennifer Burns, David Zubrow, Robert Stoddard, and Guillermo Marce-Santurio. Prioritizing alerts from multiple static analysis tools, using classification models. In *Proceedings of the 1st international workshop on software qualities and their dependencies*, pp. 13–20, 2018.

[10] Sadia Tabassum, Leandro L. Minku, and Danyi Feng. Cross-project online just-in-time software defect prediction. *IEEE Transactions on Software Engineering*, Vol. 49, No. 1, pp. 268–287, 2023.

[11] Nalin Wadhwa, Jui Pradhan, Atharv Sonwane, Surya Prakash Sahu, Nagarajan Natarajan, Aditya Kanade, Suresh Parthasarathy, and Sriram Rajamani. Core: Resolving code quality issues using llms. *Proceedings of the ACM on Software Engineering*, Vol. 1, No. FSE, July 2024.

[12] Adam Tornhill and Markus Borg. Code red: the business impact of code quality - a quantitative study of 39 proprietary production codebases. In *Proceedings of the International Conference on Technical Debt*, (TechDebt '22), p. 11–20, 2022.

[13] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E Hassan. An empirical study of the impact of modern code review practices on software quality. *Empirical Software Engineering*, Vol. 21, No. 5, pp. 2146–2189, 2016.

[14] Bogdan Vasilescu, Yue Yu, Huaimin Wang, Premkumar Devanbu, and Vladimir Filkov. Quality and productivity outcomes relating to continuous integration in github. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering*, (ESEC/FSE), p. 805–816, 2015.

[15] Carmine Vassallo, Sebastiano Panichella, Fabio Palomba, Sebastian Proksch, Harald C Gall, and Andy Zaidman. How developers engage with static analysis tools in different contexts. *Empirical Software Engineering*, Vol. 25, No. 2, pp. 1419–1457, 2020.

[16] Moritz Beller, Radjino Bholanath, Shane McIntosh, and Andy Zaidman. Analyzing the state of static analysis: A large-scale evaluation in open source software. In *Proceedings of the 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER'16)*, pp. 470–481, 2016.

[17] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. Why don't software developers use static analysis tools to find bugs? In *Proceedings of the 35th International Conference on Software Engineering (ICSE'13)*, pp. 672–681, 2013.

[18] Enas A Alikhashashneh, Rajeev R Raje, and James H Hill. Using machine learning techniques to classify and predict static code analysis tool warnings. In *2018 IEEE/ACS 15th International Conference on Computer Systems and Applications (AICCSA)*, pp. 1–8. IEEE, 2018.

[19] UV Tsiazhkorob and VN Ignatyev. Classification of static analyzer warnings using machine learning methods. In *2024 Ivannikov Memorial Workshop (IVMEM)*, pp. 69–74. IEEE, 2024.

[20] Hyunsu Kim, Mukund Raghothaman, and Kihong Heo. Learning probabilistic models for static analysis alarms. In *Proceedings of the 44th International Conference on Software Engineering (ICSE'22)*, pp. 1282—1293, 2022.

[21] Junjie Wang, Song Wang, and Qing Wang. Is there a “golden” feature set for static warning identification? an experimental evaluation. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM’18)*, 2018.

[22] Mingshuang Qing, Xiang Feng, Jun Luo, Wanmin Huang, Jingui Zhang, Ping Wang, Yong Fan, Xiuting Ge, and Ya Pan. A machine learning-based static analysis warning prioritization. In *Proceedings of the 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C’21)*, pp. 685–690, 2021.

[23] Hong Jin Kang, Khai Loong Aw, and David Lo. Detecting false alarms from automatic static analysis tools: How far are we? In *Proceedings of the 44th International Conference on Software Engineering (ICSE’22)*, p. 698–709, 2022.

[24] Mohammad Mahdi Mohajer, Reem Aleithan, Nima Shiri Harzevili, Moshi Wei, Alvine Boaye Belle, Hung Viet Pham, and Song Wang. Effectiveness of chatgpt for static analysis: How far are we? In *Proceedings of the 1st ACM International Conference on AI-Powered Software*, AIware 2024, p. 151–160, 2024.

[25] Sebastiano Panichella, Venera Arnaoudova, Massimiliano Di Penta, and Giuliano Antoniol. Would static analysis tools help developers with code reviews? In *Proceedings of the 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER’15)*, pp. 161–170, 2015.

[26] Diego Marcilio, Rodrigo Bonifácio, Eduardo Monteiro, Edna Canedo, Welder Luz, and Gustavo Pinto. Are static analysis violations really fixed? a closer look at realistic usage of sonarqube. In *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*, pp. 209–219. IEEE, 2019.

[27] 名倉正剛, 田口健介, 高田眞吾. コーディング規約違反メトリクスに基づきソフトウェア変更に対して不具合混入を予測する手法. 情報処理学会論文誌, Vol. 61, No. 4, pp. 895–907, 2020.

[28] Pascal Joos, Islem Bouzenia, and Michael Pradel. Codecureagent: Automatic classification and repair of static analysis warnings. *arXiv preprint arXiv:2509.11787*, 2025.

[29] Jeanine Romano, Jeffrey D Kromrey, Jesse Coraggio, and Jeff Skowronek. Appropriate statistics for ordinal level data: Should we really be using t-test and cohen’sd for evaluating group differences on the nsse and other surveys. In *annual meeting of the Florida Association of Institutional Research*, 第 177 巻, 2006.